

UNIVERSIDADE DE SANTIAGO DE
COMPOSTELA



ESCOLA TÉCNICA SUPERIOR DE ENXEÑARÍA

Benchmark para compresión de sinais e procesamento de imaxes en aritmética de 16 e 32 bits

Autor:

Eloi Barcón Piñeiro

Titores:

Pablo Quesada Barriuso

Javier López Fandiño

Grao en Enxeñaría Informática

Xuño 2025

Traballo de Fin de Grao presentado na Escola Técnica Superior de Enxeñaría
da Universidade de Santiago de Compostela para a obtención do Grao en
Enxeñaría Informática

Agradecementos

- Gustaríame agradecer á miña familia, especialmente á miña nai e ao meu irmán polo apoio incondicional durante toda a carreira.
- Aos meus amigos Anxo, Daniel, Martín e Breixo por axudarme facer máis liviá a miña vida académica.
- Ao meu titor e cotitor, especialmente a Pablo Quesada Barriuso por darme a idea de mudar o tema do TFG a un que me emocionou moito máis que o anterior.
- Agradecer á persoa máis importante da miña vida, o meu avó Jose Manuel Piñeiro López, que aínda que non o pode celebrar, vai ter por fin un enxeñeiro (informático pero enxeñeiro) na familia.

Resumo

O campo dos tipos de datos en punto flotante de precisión reducida, *binary16* e *bfloat16*, está dominado polas GPUs; campo onde se concentran a maioría das ferramentas, do desenvolvemento e da investigación. En comparación, a situación para estes datos en CPUs resulta máis escasa en todos estes ámbitos. Neste traballo elabórase un benchmark que permite avaliar a precisión e o rendemento destes tipos de datos nas arquitecturas de CPU x86 e ARM. Ao longo desta memoria exporase a situación actual do soporte por arquitecturas destes tipos de datos, explicaranse os programas dos que consta este benchmark e as CPUs onde se vai executar e analizaranse os resultados obtidos. Posteriormente elaboraranse unhas métricas para avaliar e contextualizar o rendemento destes datos para poder así sacar unhas conclusións da situación actual de *binary16* e *bfloat16* para CPUs.

Memoria tipo A – Índice xeral

1. Introducción	1
1.1. Descrición do problema	1
1.2. Hipótese a testar	2
1.3. Obxectivos	2
1.4. Estrutura da memoria	3
2. Estado de coñecemento do problema a abordar	5
2.1. Soporte por arquitecturas	5
2.1.1. Arquitectura x86_64	5
2.1.2. Arquitecturas baseadas en ARM	7
2.1.3. Outras arquitecturas de procesadores	7
2.2. Soporte dos compiladores	8
2.2.1. GCC	8
2.2.2. Clang	11
2.3. Tipos de datos	14
2.3.1. Tipos de datos <i>half-precision float</i>	14
2.3.2. Tipos de datos <i>brain floating point</i>	16
3. Metodoloxía	17
3.1. Materiais	17
3.1.1. Plataformas hardware	17
3.1.2. Recursos software	18
3.1.3. Docker	18
3.1.4. Librerías auxiliares para PCA	19
3.1.5. Ferramentas adicionais	19
3.2. Metodoloxía	20
3.2.1. Programas empregados	20
3.2.2. Scripts empregados	27
3.3. Retos durante o desenvolvemento	29
3.4. Métricas de rendemento	31
3.4.1. Métricas para tempos	31
3.4.2. Métricas para resultados	31

4. Probas e discusión dos resultados	33
4.1. Discusión dos resultados	33
4.1.1. Métricas de desempeño	33
4.1.2. Métricas de precisión	41
5. Conclusións e posíbeis ampliacións	49
A. Manuais técnicos	51
A.1. Estrutura de ficheiros	51
A.2. Montaxe e arranque do proxecto	54
B. Manuais de usuario	57
B.1. Compilación dos programas	57
B.2. Execución dos programas	57
B.3. Argumentos dos programas	58
B.4. Obtención de datos	59
B.4.1. Obtención dos tempos de execución	59
B.4.2. Obtención dos resultados de execución	60
B.4.3. Obtención das métricas	62
C. Licenza	65
Bibliografía	69

Índice de figuras

4.1. Speedup acadado no algoritmo axpy respecto da versión de 32 bits na arquitectura x86 cun procesador Intel.	34
4.2. Speedup acadado no algoritmo axpy respecto da versión de 32 bits na arquitectura ARM.	34
4.3. Speedup acadado no algoritmo dct respecto da versión de 32 bits na arquitectura x86 cun procesador Intel.	35
4.4. Speedup acadado no algoritmo dwt nunha dimensión respecto da versión de 32 bits na arquitectura x86 cun procesador Intel.	36
4.5. Speedup acadado no algoritmo dwt nunha dimensión respecto da versión de 32 bits na arquitectura ARM.	36
4.6. Speedup acadado no algoritmo pca, con e sen a reimplementación de lapacke_ssyev, respecto da versión de 32 bits na arquitectura x86 cun procesador Intel.	37
4.7. Speedup acadado no algoritmo pca, con e sen a reimplementación de lapacke_ssyev, respecto da versión de 32 bits na arquitectura ARM.	37

Índice de táboas

2.1. Disponibilidade de instrucións F16C, AVX512BF16 e AVX512FP16 por subconxuntos de arquitecturas x86 [10].	7
2.2. Flags de GCC para compilación de programas FP16 e BF16 por arquitecturas	11
2.3. Flags de Clang para compilación de programas FP16 e BF16 por arquitecturas	13
4.1. Erro absoluto medio, respecto aos resultados de <i>float</i>	41
4.2. Erro cadrático medio, respecto aos resultados de <i>float</i>	43
4.3. Raíz do erro cadrático medio normalizada, respecto aos resultados de <i>float</i>	45
4.4. Coeficiente de correlación de Pearson, respecto aos resultados de <i>float</i>	46

Capítulo 1

Introdución

No estado actual da computación de alto rendemento e no campo da IA, a eficiencia no procesamento e a redución do consumo enerxético impulsaron o interese por crear e empregar tipos de datos de precisión reducida. Introducíronse os datos en punto flotante de media precisión (*half-precision float*), tamén coñecido como *binary16*, segundo o estándar IEEE 754-2008 [1], e os *brain floating point*, *bfloat16*, que foi desenvolto por Google Brain [2, 3]. Estes datos creáronse para optimizar tanto a velocidade de cálculo como o uso de memoria en aplicacións intensivas especialmente nas relacionadas coa aprendizaxe automática e a inferencia de redes neuronais.

Estes tipos de datos ofrecen un menor consumo de memoria pero limitan a exactitude dos cálculos se os comparamos cos tipos de datos en punto flotante de precisión simple (tamén chamados *single-precision float*). Nos últimos anos estes tipos de datos experimentaron un aumento de uso polo auxe da intelixencia artificial e os modelos de linguaxe de grande escala (LLMs). A maior parte do desenvolvemento e avance neste campo realizouse en torno ás GPU, e en concreto ás de Nvidia, polo que o campo das CPUs está menos desenvolto. Con todo, as recentes inclusións de extensións como AVX-512BF16 e AVX-512FP16 en Intel ou ARmv8.6-A empezan a abrir escenarios para o soporte nativo destes tipos de datos en CPU.

1.1. Descrición do problema

Este traballo aborda a crecente relevancia dos tipos de datos flotantes de media precisión (*half-precision float*) e *brain floating point* no ámbito da programación moderna especialmente cos avances da tecnoloxía de hardware. Sábese que os tipos de datos en media precisión ocupan menos memoria e poden representar menos información que os tipos de datos flotantes de precisión simple, pero non está claro se esta perda de precisión pode ser compensada pola mellora de rendemento polo que, neste traballo vaise determinar como varía o rendemento

destes tipos de datos con respecto aos de simple precisión.

O obxectivo principal do traballo é a elaboración de programas en linguaxe C que empreguen tipos de datos de 32 bits, e versións equivalentes destes programas que empregan tipos de datos de 16 bits, que conformen un benchmark para avaliar o rendemento tanto en tempo de execución como en perda de información entre versións. Para ter unha visión da situación entre arquitecturas as probas realizaranse tanto nunha arquitectura x86 de 64 bits (tamén chamada `x86_64`) como nunha arquitectura ARMv8.2-A [4]. Posteriormente se realizarán comparativas que abranguerán versións diferentes nunha mesma arquitectura e a mesma versión en arquitecturas distintas.

1.2. Hipótese a testar

Formúlanse as seguintes hipóteses para probar coa realización deste traballo:

- H1: A mellora de rendemento é máis notábel en arquitecturas que teñen soporte nativo para os tipos de datos *half-precision float* e *brain floating point*.
- H2: A perda de precisión por empregar estes tipos de datos é compensada suficientemente pola mellora de rendemento.
- H3: A mellora de rendemento é dependente da cantidade de datos empregados e das operacións efectuadas.
- H4: Non se observa unha diferenza significativa na perda de precisión entre a execución nativa e non nativa destes tipos de datos.

1.3. Obxectivos

De forma xenérica, plantexouse como obxectivo principal deste Traballo de Fin de Grao a **análise do rendemento e precisión do uso de tipos de datos *half-precision float* e *brain floating point***, para isto implementáronse varios algoritmos de distinta complexidade cada un con diferentes versións, unha por cada tipo de dato.

Tamén se tomou como obxectivo a **comparativa de resultados entre as arquitecturas de procesadores x86 de 64 bits e ARM de 64 bits**, empregando x86 como base para a comparativa con ARM. Isto permitirá verificar o funcionamento nunha arquitectura con soporte nativo e discernir tamén algunha situación na que o uso non nativo poda ser relevante.

1.4. Estrutura da memoria

Este Traballo de Fin de Grao pódese situar dentro do Tipo A segundo o Regulamento do Traballo de Fin de Grao do Grao en Enxeñaría Informática pola USC, na súa última modificación aprobada pola Xunta de Escola da ETSE en Xaneiro de 2025. É dicir, a súa finalidade é o «Desenvolvemento dunha idea [...] dun Sistema Informático que constitúa unha contribución ás técnicas da Informática» [5] e neste caso no campo dos tipos de datos de *half-precision float* e *brain floating point* para a linguaxe de programación C.

Esta memoria estrutúrase nos seguintes capítulos, cada un cuns obxectivos concretos, que buscan abarcar todos os coñecementos precisos para poder entender este traballo:

- Capítulo 1: Introducción aos temas que van ser tratados ao longo do traballo así como aos obxectivos propostos para o mesmo.
- Capítulo 2: Repaso da situación actual dos tipos de datos empregados e das ferramentas, tanto hardware como software, que se poden utilizar.
- Capítulo 3: Presentación dos materiais e ferramentas empregadas para o desenvolvemento do traballo, a metodoloxía seguida para obter os resultados e as métricas de rendemento para avalialos.
- Capítulo 4: Descrición das probas realizadas destacando como se realizan e que conxuntos de datos se empregan. Este capítulo tamén terá unha discusión dos resultados obtidos mediante as probas anteriormente mencionadas.
- Capítulo 5: Síntese das conclusións extraídas dos resultados obtidos. Tamén se comentan posíbeis ampliacións ou melloras respecto ao traballo efectuado.

Capítulo 2

Estado de coñecemento do problema a abordar

Neste capítulo realizarase unha contextualización do estado actual das operacións con datos en punto flotante de precisión reducida, *binary16* e *bfloat16*, e tamén da situación concreta para a linguaxe de programación C. Esta contextualización será tanto de hardware, para arquitecturas e para instrucións de procesador; como de software, para compiladores e tipos de datos admitidos.

2.1. Soporte por arquitecturas

O soporte para operacións con tipos de datos *half-precision float* e *brain floating point* por arquitecturas é moi variado posto que cada unha ten as súas propias instrucións, e versións e non, todas presentan soporte completo para ambos tipos de datos. Como no traballo desenvolto as arquitecturas nas que se incide son as de procesadores x86_64 e ARM serán estas nas que máis se incida neste apartado, pero tamén se mencionarán outras arquitecturas de procesadores para dar unha idea da situación xeral.

2.1.1. Arquitectura x86_64

A arquitectura x86_64, dominante en servidores e escritorios, ten un soporte heteroxéneo para tipos de media precisión. Aínda que Intel e AMD comparten a mesma ISA (Instruction Set Architecture), a implementación de extensións relacionadas con *binary16* (FP16) e *bfloat16* (BF16) varía entre fabricantes e xeracións de procesadores. A continuación, analízanse os principais aspectos técnicos.

Extensións relevantes

- Instrucións **F16C** (orixinalmente CVT16): son unha extensión das instrucións do núcleo SSE de 128 bits introducidas no 2009 que permite conversións entre *half-precision* e *single-precision* en ambas direccións, sen ofrecer operacións aritméticas directas para *half-precision* [6]. Resulta útil para axudar á compatibilidade entre os tipos de datos, pero limitada en eficiencia porque só se tratan de operacións de conversión. En **AMD** estas introducíronse por primeira vez na microarquitectura **Jaguar**, mantendo o soporte ata a actualidade. En **Intel** a súa incorporación tivo lugar na microarquitectura **Ivy Bridge** e mantendo tamén soporte ata a actualidade.
- Instrucións **AVX512FP16** (Advanced Vector Extensions 512 - FP16) [7]: Parte das instrucións AVX512 que permiten realizar operacións aritméticas con datos *half-precision float*. Para procesadores **AMD** aínda non hai soporte oficial pero a futura arquitectura **Zen 6** prevese que terá soporte para estas instrucións [8, 9] e por tanto poderá realizar operacións aritméticas con estes tipos de datos, pero non hai confirmación oficial por parte de AMD. A súa dispoñibilidade en procesadores **Intel** é máis complexa posto que tecnicamente existe soporte en procesadores con microarquitectura Alder Lake, aínda que non hai soporte en todos os procesadores desta microarquitectura [10]. A partir da microarquitectura **Sapphire Rapids** é cando se introduce propiamente o soporte para estas instrucións en procesadores para servidor como o Intel Xeon Scalable 4^a xeración [7]. O soporte para procesadores de consumo é máis complexo posto que Intel desactivou por defecto estas instrucións na arquitectura Alder Lake [11] e só algúns procesadores poden volver habilitalas[12].
- Instrucións **AVX512BF16** (Advanced Vector Extensions 512 - BF16) [13]: Parte das extensións AVX512, que permiten realizar operacións aritméticas con datos *brain floating point*. En procesadores **AMD** o soporte destas extensións está dispoñíbel dende a microarquitectura **Zen 4** [10], permitíndolle a todos os procesadores que teñan polo menos esta microarquitectura operar con datos *bfloat16*. No caso de **Intel** a situación é similar ao caso anterior, introducíronse por primeira vez para a microarquitectura **Cooper Lake**. Nas seguintes microarquitecturas os procesadores deixaron de incluír estas instrucións (veñen desactivadas por defecto en Alder Lake) ata que **Sapphire Rapids** volveu engadir esta extensión [10]. Aínda que estas instrucións permiten realizar operacións aritméticas nativas con datos *bfloat16*, abarcan un conxunto de operacións menor que as instrucións AVX512FP16 [13, 14]

O resumo do soporte por microarquitecturas pódese consultar na Táboa 2.1:

Subconxunto	F16C	AVX512BF16	AVX512FP16
Cooper Lake (2020)	Si	Si	Non
Ice Lake (2019)	Si	Non	Non
Tiger Lake (2020)	Si	Non	Non
Rocket Lake (2021)	Si	Non	Non
Alder Lake (2021)	Si	Parcial	Parcial
Zen 4 (2022)	Si	Si	Non
Sapphire Rapids (2023)	Si	Si	Si
Zen 5 (2024)	Si	Si	Non

Táboa 2.1: Disponibilidade de instrucións F16C, AVX512BF16 e AVX512FP16 por subconxuntos de arquitecturas x86 [10].

2.1.2. Arquitecturas baseadas en ARM

ARM permite operar con tipos de datos *binary16* e *bfloat16* dende a arquitectura AArch64 ou ARM64 [15] pero hai matices respecto ao soporte completo destes tipos de datos. En xeral poden traballar con programas que empreguen estes tipos de datos pero dependendo da versión pode ser que se emule este funcionamento se non hai soporte (afondarase máis nisto na Sección 2.2).

IEEE 754 de 16 bits (binary16) format

ARM presenta soporte para almacenamento deste tipo de dato con ARMv8-A (primeira versión en empregar AArch64), en concreto é antes de ARMv8.2-A [15] cando se engade o soporte. É propiamente en **ARMv8.2-A** cando se engade procesamento de datos *half-precision float* de xeito opcional para os procesadores [15] o que permite realizar operacións con estes tipos de datos sen ter que facer conversións intermedias, é dicir, permite soporte completo para *binary16*.

Brain floating-point format (bfloat16)

Ao contrario do que sucedeu cos tipos de datos *half-precision float*, no caso dos tipos de datos *brain floating point* o seu soporte está dispoñíbel a partir de ARMv8.2-A só para almacenamento [16]. É a partir de **ARMv8.6-A** onde se engade soporte para a realización nativa de operacións aritméticas con estes tipos de datos, aínda que as extensións *bfloat16* para SVE (Scalable Vector Extension) e Neon son principalmente para uso en aprendizaxe profunda [15].

2.1.3. Outras arquitecturas de procesadores

- **RISC-V**: O soporte para *half-precision float* está incluído nas extensións **Zfh** e **Zfhmin** [17], para o tipo de dato *brain floating point* son necesarias

as extensións **Zfbfmin**, **Zvfbfmin**, e **Zvfbfwma** [18, 19]. RISC-V en Linux recibirá soporte para máis instrucións *bfloat16* co kernel 6.15 [20].

- **Power ISA:** O soporte hardware para *half-precision float* incorporouse con VSX e coa extensión SVP64, ambas introducidas en **Power ISA v.3.1**[21]. *bfloat16* recibiron soporte coa familia de procesadores **Power10**, que emprega instrucións de Power ISA v.3.1 [22].
- **LoongArch:** En **LoongArch v1.1** hai soporte para operacións que empregan *half-precision float* mediante as **instrucións con sufixo .H** [23].

2.2. Soporte dos compiladores

O papel dos compiladores resulta fundamental para aproveitar ao máximo as capacidades do hardware relacionadas co soporte de tipos de datos en punto flotante de media precisión (FP16) e *brain floating point* (BF16). Estas ferramentas son as encargadas de realizar a tradución do código fonte escrito en linguaxes de alto nivel, como C no noso caso, a instrucións optimizadas que poden executarse eficientemente nos sistemas, aproveitando así as capacidades do hardware descritas anteriormente. Dado que o traballo desenvolvido céntrase na linguaxe C, unha das linguaxes de medio nivel máis empregadas en programación [24], é chave analizar como os compiladores máis relevantes do ecosistema, GCC (GNU Compiler Collection) e Clang, implementan o soporte para estes formatos de datos. Ambos compiladores ofrecen extensións específicas para manipular tipos de datos como FP16 e BF16, pero as diferenzas nos seus enfoques de optimización e na xestión de compatibilidade entre arquitecturas poden ter un impacto significativo no rendemento e na portabilidade do código. Nesta sección afondarase máis no compilador GCC por tratarse do empregado para a compilación dos programas en C deste traballo pero tamén se informará de Clang debido á súa crecente relevancia.

2.2.1. GCC

GNU Compiler Collection ou GCC [25] é, como o seu nome indica, unha colección de compiladores creada polo proxecto GNU que orixinalmente só compilaba a linguaxe de programación C e posteriormente engadiu soporte para outras linguaxes, como C++ ou Fortran, pero nesta sección ímonos enfocar no relativo a C. É o compilador máis antigo e co maior soporte para arquitecturas dos compiladores de C [25], de feito ten soporte para todas as arquitecturas de CPUs mencionadas anteriormente (incluíndo LoongArch dende a versión 12.1 [26]).

Pódese ver un resumo dos flags precisos para a compilación na Táboa 2.2

Compilación en x86

half-precision float: GCC recomenda empregar o tipo de dato `_Float16` [27, 28] (afondaremos nel na Sección 2.3) para traballar de xeito nativo con estes tipos de datos en x86 o programa ten que compilarse coa flag `-mavx512fp16` para que empregue instrucións especializadas (AVX512-FP16). O soporte para compilar con este flag foi engadido na **versión 12 de GCC** [29]. Se o usuario compilou co flag `-mavx512fp16` xéranse instrucións “AVX512FP16”, sen fallback nin emulación por software; se o procesador non as soporta, as instrucións xerarán unha excepción de instrución ilegal en tempo de execución. Se non se compila con este flag, as operacións serán emuladas, teña soporte ou non o procesador [30].

Para a compilación dun programa que vai ser emulado pódese empregar o flag `-fexcess-precision=` para axustar o rendemento ou a precisión, este flag ten os seguintes valores:

- `-fexcess-precision=fast:` opción por defecto cando non hai soporte nativo. Todo o resultado intermedio almacénase promocionado a *float* e só hai conversións cando se asignan os valores para maximizar o rendemento [31]. Isto pode ocasionar inconsistencias co resultado dunha execución nativa.
- `-fexcess-precision=16:` Ao rematar cada operación o valor redondéase de novo a 16 bits para intentar minimizar o erro, isto provocará un maior custo no rendemento. Pode reducirse co uso das instrucións F16C, activadas compilando co flag `-mf16c` [32].

brain floating point: para traballar con este formato en C, GCC só dispón do tipo intrínseco `__bf16` (afondaremos nel na Sección 2.3), que se apoia na especificación do x86_64 psABI [33]. Non existe como tipo definido polo estándar ISO C (non hai un “_FloatBF16” análogo a `_Float16`). O soporte para compilar con este intrínseco foi engadido na **versión 13 de GCC** [34]. Para compilar programas que empreguen instrucións especializadas (AVX512-BF16) hai que empregar o flag `-mavx512bf16`, en caso contrario o comportamento será emulado, podendo empregar o flag `-fexcess-precision=`, igual que pasaba con *half-precision float*.

Se o usuario compilou co flag `-mavx512bf16` sen soporte, igual que con *half-precision float*, saltará unha excepción de instrución ilegal en tempo de execución. En programas moi sinxelos é posíbel que non suceda porque o set de instrucións para *bfloat16* é máis reducido e pode ser que non se empregue unha instrución non recoñecida [13, 14].

Compilación en ARM

half-precision float: En arquitecturas ARM, GCC soporta *half-precision float* a través de `__fp16` (afondaremos nel na Sección 2.3) e `_Float16`. En arquitecturas ARM hai que empregar o flag `-mfp16-format=`, que pode ter dúas opcións: `-mfp16-format=ieee` que emprega o formato IEEE 754-2008 para os datos *half-precision float* e `-mfp16-format=alternative` que emprega o formato alternativo de ARM [30]. En AArch64 GCC só soporta o formato IEEE 754-2008 polo que este flag deixa de ser necesario [30].

ARM xera instrucións para conversión entre `__fp16`, ou `_Float16`, e `float` (ou `double`) ao compilar con opcións que seleccionan a FPU que as proporciona como `-mfp16=neon-fp16 -mfloat-abi=softfp`. Para activar a extensión FP16 para AArch64 o programa ten que ser compilado co flag `-march=arch+fp16` [35] (Por exemplo para ARMv8.2-A (mínima soportada nativa) sería `-march=armv8.2-a+fp16`), engadido na **versión 9 de GCC** [36]. Para empregar `__fp16` o programa a compilar precisa ademais a biblioteca propia de ARM `arm_fp16.h` [37], que non pode empregarse noutras arquitecturas. Compilar con `-march=arch+fp16` en versións sen soporte, posíbel só dende GCC 14 [38], xerará código con instrucións que os procesadores non serán capaces de interpretar, o soporte para *half-precision float* foi introducido en ARMv8.2-A. Compilar un programa sen este flag permite executar os programas igualmente aínda que mediante emulación.

Para a compilación dun programa que vai ser emulado pódese empregar o flag `-fexcess-precision=` [31] (que funciona exactamente igual que para x86) para axustar o rendemento ou a precisión.

brain floating point: para traballar con este tipo de dato en C, GCC só dispón do intrínseco `__bf16` e para usalo en ARM é preciso ter a biblioteca `arm_bf16.h` [37] no programa a compilar. Dito programa ten que ser compilado en AArch64 co flag `-march=arch+bf16` [35] (por exemplo para ARMv8.6-A (mínima soportada) sería `-march=armv8.6-a+bf16`), esta opción está activada por defecto para `-march=armv8.6-a` [35]. Este flag foi introducido na **versión 10 de GCC** [39]. Empregar este flag con arquitecturas anteriores a Armv8.2-A non está soportado.

De igual xeito que con `__fp16`, compilar con este flag en versións de ARM que non o soportan xerará código con instrucións que os procesadores non serán capaces de interpretar, posto que o soporte para *bfloat16* foi introducido en ARMv8.6-A. Compilar sen este flag permite executar os programas igualmente aínda que mediante emulación, podendo empregar o flag `-fexcess-precision=`, igual que pasaba con *half-precision float*.

Arquitectura	FP16	BF16
x86 nativa	-mavx512fp16	-mavx512bf16
ARM nativa	-march=armv8.2-a+fp16	-march=armv8.6-a+bf16
Emulación	-fexcess-precision=	-fexcess-precision=

Táboa 2.2: Flags de GCC para compilación de programas FP16 e BF16 por arquitecturas

2.2.2. Clang

Clang é un compilador front-end de código aberto para C, entre outras linguaxes de programación, deseñado como alternativa a GCC e que emprega LLVM como o seu back-end. Foi lanzado en 2007 e no seu desenvolvemento están implicadas varias compañías como Apple ou Google [40].

Clang presenta soporte para 3 tipos de datos de precisión reducida: `__fp16`, `_Float16` e `__bf16`. O soporte destes datos depende da arquitectura de destino, será nativo se o procesador pode realizar operacións aritméticas sobre el, en caso contrario o compilador emulará a aritmética promovendo a float.

Pódese ver un resumo dos flags precisos para a compilación na Táboa 2.3

Compilación en x86

half-precision float: `__fp16` está soportado nativamente en x86 sempre, mentres que `_Float16` o está de xeito emulado se o procesador conta con instrucións SSE2, se ademais conta con instrucións AVX512FP16 o soporte será nativo [41]. Clang recomenda empregar `_Float16` en troques de `__fp16` porque foi definido polo comité de estándares de C [41]. Respecto a `__fp16`, a propia documentación de Clang especifica que era orixinalmente unha extensión de ARM, emprega o formato *binary16* e nas operacións aritméticas é promovido a *float* [41]. O soporte para `_Float16` foi introducido en **LLVM 15**, o que se traduce en que **Clang 15** tamén o soporta [42], porén o soporte para `__fp16` foi introducido en **LLVM 14** [29], é dicir, **Clang 14**. Para xerar código que funcione de xeito nativo con estes tipos de datos habería que compilar co flag `-mavx512fp16` [43]. É importante salientar que se o programa emprega `__fp16` haberá que compilar co flag `-Xclang -fnative-half-arguments-and-returns` se se quere que algunha función o teña como argumento ou devolva este tipo de dato [44]. Se o usuario compilou con `-mavx512fp16` xéranse instrucións “AVX512FP16” e se o procesador non as soporta, as instrucións xerarán unha excepción de instrución ilegal en tempo de execución.

Cando se compila sen este flag a execución será emulada, para o cal Clang realizará a aritmética en float, truncando resultados intermedios a 16 bits

unicamente cando sexa necesario (*excess precision*), para controlarlo pódese empregar o flag `-ffloat16-excess-precision=` [41] e ten 3 opcións:

- `-ffloat16-excess-precision=none`: comportamento por defecto para a emulación. Só trunca cando é estritamente necesario, é dicir, en asignacións ou casts. É a que xera a execución con mellor rendemento.
- `-ffloat16-excess-precision=standard`: realiza un truncamento tras cada operación (similar a `-fexcess-precision=16` en GCC) resultando nunha emulación estrita do funcionamento nativo a costa dun menor rendemento. Para mellorar este rendemento pódese compilar adicionalmente con `-mf16c` [41] que permite empregar as instrucións F16C.
- `-ffloat16-excess-precision=fast`: realiza algún truncamento máis que `none` pero non tantos como `standard` para buscar un equilibrio entre fidelidade dos resultados e rendemento, o que o deixa nun punto medio dos dous casos. Tamén pode empregar o flag `-mf16c` [41].

brain floating point: O tipo de dato `__bf16` está soportado en x86 se o procesador conta con instrucións SSE2, de xeito emulado, se ademais conta con instrucións AVX512BF16 este soporte será nativo. O soporte para `__bf16` foi introducido en **LLVM 16**, é dicir, **Clang 16** [34]. Para xerar código que funcione de xeito nativo con estes tipos de datos habería que compilar o programa co flag `-mavx512bf16` [45]. Se o usuario compilou con este flag xéranse instrucións “AVX512BF16” e se o procesador non as soporta, as instrucións xerarán unha excepción de instrución ilegal en tempo de execución. En programas moi sinxelos é posíbel que non suceda [13, 14].

Cando se compila sen este flag a execución será emulada, para o cal Clang empregará *excess precision*, este comportamento pode ser controlado mediante o flag `-fbfloat16-excess-precision=`, que ten as mesmas versións que `-ffloat16-excess-precision=` indicado anteriormente.

Compilación en ARM

half-precision float: ao igual que en x86 temos os tipo de datos `__fp16` e `_Float16`. `__fp16` está soportado en tanto ARM de 32 bits como AArch64 e precisa engadir a biblioteca `arm_fp16.h` [37] ao programa. `_Float16` está soportado de xeito emulado en AArch64, e nalgúns versións de ARM de 32 bits, se ademais se trata de ARMv8.2-A ou unha versión superior este soporte será nativo.

Para poder realizar a execución nativa dos programas que empregan estes tipos de datos hai que compilar mediante o flag `-march=arch+fp16` [46, 47] (por exemplo para ARMv8.2-A, mínima con soporte para o tipo de dato, sería `-march=armv8.2-a+fp16`). De xeito similar ao que pasaba con GCC,

compilar nunha arquitectura de destino que non soporta ditas instrucións, ocasionará que as instrucións xeren unha excepción de instrución ilegal en tempo de execución. Destacar que cando empregamos o tipo de dato `__fp16` as operacións aritméticas non serán executadas nunca de xeito nativo porque é un tipo de dato só para almacenamento (afondaremos nisto na Sección 2.3). As operacións, ao igual que cando non hai soporte nativo non se emprega para realizar operacións aritméticas, as cales serán executadas en *float* (ou ás veces *double*) e ao rematar o resultado será transformado de volta a `__fp16` [48]. A diferenza con respecto a emulación é que non se ven afectados polos flags de *excess precision*.

Cando se compila sen o flag `-march=arch+fp16` o código xerado non emprega instrucións específicas, a execución será emulada, polo que Clang empregará *excess precision*. Este comportamento pode ser controlado mediante o flag `-ffloat16-excess-precision=` como en x86.

brain floating point: do mesmo modo que en x86 temos o tipo de dato `__bf16` que está soportado en ARMv8.2-A, e versións posteriores de xeito emulado, se ademais se trata de ARMv8.6-A ou unha versión superior será nativo, non emulado. Para empregar este tipo de dato o programa necesitará engadir a biblioteca `arm_bf16.h` [37]. Para poder realizar a execución nativa dos programas que empregan estes tipos de datos hay que compilar mediante o flag `-march=arch+bf16` [46, 47] (por exemplo para ARMv8.6-A (mínima soportada) sería `-march=armv8.6-a+bf16`). Destacar que, de xeito similar ao que pasaba con GCC, o compilador non prevén ao usuario de empregar este flag cunha arquitectura de destino que non soporta ditas instrucións, isto ocasionará que as instrucións xeren unha excepción de instrución ilegal en tempo de execución.

Cando se compila sen o flag `-march=arch+bf16` o código xerado non emprega instrucións específicas, a execución será emulada, polo que Clang empregará *excess precision*. Este comportamento pode ser controlado mediante o flag `-fbfloat16-excess-precision=` como en x86.

Target	FP16	BF16
x86 nativa	<code>-mavx512fp16</code>	<code>-mavx512bf16</code>
ARM nativa	<code>-march=armv8.2-a+fp16</code>	<code>-march=armv8.6-a+bf16</code>
Emulación	<code>-ffloat16-excess-precision=</code>	<code>-fbfloat16-excess-precision=</code>

Táboa 2.3: Flags de Clang para compilación de programas FP16 e BF16 por arquitecturas

2.3. Tipos de datos

Na computación de alto rendemento e en áreas como a aprendizaxe automática, o uso de aritmética de precisión reducida resulta fundamental para optimizar tanto o rendemento computacional como o consumo de recursos. Neste contexto os tipos de datos *half-precision float* e *brain floating point* gañaron relevancia, ofrecendo alternativas prometedoras para aplicacións onde a redución de precisión ou o aumento de rendemento resultan vantaxosas.

Esta sección analiza as características e a evolución destes tipos de datos, salientando a distinción funcional entre aqueles deseñados unicamente para almacenamento e aqueles outros que permiten realizar operacións aritméticas directamente sen conversións intermedias. Resaltarase o impacto desta diferenciación na eficiencia e na fidelidade dos cálculos, especialmente no relativo ao comportamento da *excess precision* [49] segundo a linguaxe de programación C.

Máis adiante abordarase, de xeito diferenciado, a implementación de cada un destes tipos de datos. Esta revisión do estado do coñecemento pretende ofrecer un marco que facilite a interpretación das vantaxes e limitacións de cada enfoque en distintos entornos de hardware.

2.3.1. Tipos de datos *half-precision float*

Tamén denominado media precisión, *FP16* ou *float16*, é un formato de número binario de punto flotante que ocupa 16 bits na memoria do ordenador. Está destinado principalmente a almacenamento de valores en punto flotante cando a precisión non é esencial, destacando procesado de imaxes e redes neuronais [50]. Case todos os usos modernos seguen o estándar IEEE 754-2008 [1] onde o formato en base 2 de 16 bits é coñecido como *binary16* e emprega 5 bits para o expoñente.

O estándar IEEE 754 especifica o seguinte formato para *binary16*:

Bit de signo: 1 bit.

Ancho do expoñente: 5 bits.

Precisión significativa: 11 bits (10 bits almacenados explicitamente).

O expoñente está codificado empregando unha representación “offset-binary”, estando o 0 desprazado 15, deixando por tanto -14 como expoñente mínimo e 15 como expoñente máximo. O valor mínimo enteiro normal representábel é $2^{-14} \cdot (1 + \frac{0}{1024}) \approx 6,103515625 \cdot 10^{-5}$, e o máximo é $2^{15} \cdot (1 + \frac{1023}{1024}) \approx 65504,0$.

En C este tipo de dato corresponde con `_Float16` en x86 mentres que en ARM corresponde con `_Float16` e `__fp16` compilado co flag `-mfp16-format=ieee` e representan valores *binary16*. Se nunha arquitectura ARM (anterior a AArch64 [30]) compilamos con `-mfp16-format=alternative` este empregará o formato alternativo de ARM que é exactamente igual a *binary16* pero non ten codificación

para o infinito nin para NaNs (elimina o caso especial dun valor con expoñente 31) o que lle permite representar ata 131008. A partir de AArch64 xa non é posíbel, eliminando por tanto a necesidade do flag `-mfp16-format`.

Respecto á finalidade dos tipos de datos, cada uno deles ten un fin diferente:

— ***fp16***: é un tipo de dato que pertence exclusivamente ás extensións de linguaxe ARM (ACLE) [48, 30] (e recoñecido polo compilador Clang en base á extensión de ARM). Este tipo emprégase como formato de almacenamento, é dicir, a súa principal vantaxe é que permite gardar os datos en media precisión para reducir a memoria que ocupan e o ancho de banda (respecto a *float*). Porén, as operacións aritméticas que se realicen con este tipo de dato non se executan realmente con datos *binary16*, senón que o compilador convérteos a unha maior precisión (como *float*) para realizar as operacións [48, 41] (traballando con *excess precision* [49]). Este tipo de dato é capaz de aproveitar as instrucións nativas de conversión cara e dende *float* compilado cos flags `-march=arch+fp16`, para ARM e `-mf16c`, para x86 só se compila con Clang.

— ***Float16***: este tipo de dato foi introducido no estándar ISO/IEC TS 18661-3 [28] de C (polo que pode empregarse en todas as arquitecturas) para facilitar as operacións aritméticas en formato *half-precision* de xeito nativo, sempre que o hardware e o compilador o permitan. Este tipo de dato é capaz de almacenar e de realizar operación cos datos polo que, compilado co flag `-mavx512fp16` en x86 ou co flag `-march=arch+fp16` en ARM, pode facer todas as operacións sen facer conversións intermedias. En arquitecturas x86 sen instrucións AVX512FP16 e en arquitecturas ARM anteriores a ARMv8.2-A as operacións deberán ser emuladas, mediante *excess precision* ao non poder executar as instrucións que xestionarían este tipo de dato nativamente. Se o programa está compilado cos flags `-fexcess-precision=16`, en GCC, ou `-ffloat16-excess-precision=standard` en Clang, ao finalizar cada operación aritmética o resultado converterase de volta a 16 bits. Se non se emprega este flag, as conversións só se realizarán cando se faga a asignación de valores neste tipo de dato, conversión non afectada polo flag anteriormente mencionado.

A partir da versión 14 de GCC, o tipo de dato `_Float16` conta con soporte independente de SSE2 en arquitecturas x86 [38], ocasionando un comportamento diferente do indicado anteriormente. Sen SSE2 este tipo de dato será tratado como só de almacenamento e o compilador dará erros cando se empregue doutro xeito (non ten soporte para aritmética). Tampouco permitirá que as funcións o empreguen como argumento nin como valor de retorno.

2.3.2. Tipos de datos *brain floating point*

Tamén *bfloat16* é un formato de número binario que ocupa 16 bits na memoria do ordenador. Este formato é unha versión abreviada do formato de punto flotante de precisión simple de IEEE 754 (*binary32*) coa intención de acelerar a aprendizaxe automática. Foi desenvolvido por un grupo de investigación de intelixencia artificial de Google [3] e segue o seguinte formato:

Bit de signo: 1 bit.

Ancho do expoñente: 8 bits.

Precisión significativa: 8 bits (7 almacenados explicitamente).

É unha versión acortada de *binary32* (float de 32 bits) o que permite conversións rápidas a e desde *binary32*, mantén os mesmos bits de expoñente e reduce os do campo do significando, que poden ser reducidos por truncamento ou calquera outro método de redondeo, ignorando o caso de NaN [2].

O expoñente está codificado cunha representación *offset-binary*, estando o 0 desprazado 127, deixando por tanto -126 como expoñente mínimo e 127 como expoñente máximo. O valor mínimo enteiro normal representábel é $2^{-126} \cdot \left(1 + \frac{0}{2^{23}}\right) \approx 1,17549435 \times 10^{-38}$, e o máximo $2^{127} \cdot \left(1 + \frac{2^{23}-1}{2^{23}}\right) \approx 3,4028235 \times 10^{38}$.

En linguaxe C este tipo de dato corresponde con `__bf16` e é compatible con x86 e con ARM aínda que non pertenza ao estándar de C. Clasifícase como *SSE-class* no *x86_64 System V psABI* [33] o que lle permite pasarse e retornarse exactamente igual que un *float*, de xeito que calquera procesador x86_64 pode executar programas que o empregue de xeito nativo ou non.

O uso deste tipo de dato en hardware con instrucións que permiten a execución nativa do mesmo pode mellorar significativamente o rendemento en frameworks coma TensorFlow ou PyTorch, e en xeral en aprendizaxe automática [16].

Para executar este tipo de dato de xeito nativo hai que compilar o programa con flag `-mavx512bf16` en x86 e `-march=arch+bf16` en ARM. En caso de non empregar estes flags a execución será emulada e o funcionamento por defecto será mediante “excess precision”, comportamento que pode modificarse cos flags `-fexcess-precision=16`, en GCC, ou `-ffloat16-excess-precision=standard` en Clang. Cabe lembrar que tal como se comentou previamente o compilador ten que recoñecer este tipo de dato para poder usalo.

A partir da versión 14 de GCC, o tipo de dato `__bf16` conta con soporte independente de SSE2 en arquitectura x86 [38], ocasionando un comportamento diferente do indicado anteriormente. Sen SSE2 este tipo de dato será tratado como só de almacenamento e o compilador dará erros cando se empregue doutro xeito (non ten soporte para aritmética). Tampouco permitirá que se pase a funcións como argumentos ou que estas o empregue como valor de retorno.

Capítulo 3

Metodoloxía

Neste capítulo preséntanse as ferramentas e plataformas hardware e software empregadas para a elaboración dos programas e a realización das execucións e posterior obtención dos resultados. Tamén se inclúe toda información relevante para poder reproducir os resultados, analízalos con rigor e, finalmente, confirmar ou rexeitar as hipóteses expostas.

3.1. Materiais

Esta sección está adicada á exposición dos materiais informáticos empregados para levar a cabo este traballo. En concreto, describiremos o hardware empregado para elaborar e probar os programas, os recursos software e outras tecnoloxías que se teñan empregado. Tamén mencionaremos ferramentas adicionais que non están directamente ligadas cos resultados finais pero que tamén foron de utilidade durante o traballo.

3.1.1. Plataformas hardware

Dado que o traballo está centrado na comparativa entre tipos de datos para a avaliación do seu rendemento, escolléronse dúas plataformas hardware diferenciadas para ter unha imaxe máis ampla da situación actual.

A primeira plataforma hardware é un ordenador portátil MSI Prestige 14 A11SCX, que conta cun procesador Intel i7-1185G7 e cunha memoria RAM de 16 GB LPDDR4, que servirá como a plataforma para a realización de probas e execucións relativas á arquitectura x86_64. Como sistema operativo conta coa distribución de Linux chamada Ubuntu na versión 24.04.2.

A outra plataforma hardware coa que se conta é un kit Jetson AGX Orin, que conta cun procesador ARM Cortex A78AE e cunha memoria RAM de 64 GB LPDDR5, que servirá como plataforma para a realización de probas e execucións relativas á arquitectura ARM. Como sistema operativo conta coa distribución de

Linux chamada Ubuntu na versión 22.04.5.

Ambos equipos serán empregados tanto para o desenvolvemento e depuración como para a execución dos programas.

3.1.2. Recursos software

Nesta sección aportarase toda a información dos recursos software que teñen unha implicación directa na execución dos programas dos que se imos obter os datos para poder crear e avaliar as métricas.

Linguaxes de programación

Para a parte de desenvolvemento do traballo empregáronse diferente linguaxes de programación en función da tarefa a realizar. Para os programas principais dos que extraer os datos empregouse a linguaxe C, para unificar e centralizar a compilación e execución dos programas empregouse Bash e para extracción de datos e elaboración das métricas empregouse Python.

Compiladores

Para este traballo empregouse o compilador GNU Compiler Collection, GCC, para a compilación de todos os programas que se queren empregar. A versión escollida para este traballo foi a 14.2 por ser a primeira versión deste compilador que permite que os tipos de datos `_Float16` e `__bf16` teñen soporte independente de SSE2 en x86[38], permitindo ampliar o experimento nun futuro.

Para o desenvolvemento en arquitecturas x86, empregouse adicionalmente o compilador cruzado baseado en GCC `aarch64-linux-gnu-gcc`, que permite compilar programas destinados a ARM de 64 bits dende arquitecturas x86 e ARM. Emprégase a versión 14, `aarch64-linux-gnu-gcc-14` [51], para que coincida coa empregada en GCC e ter exactamente as mesmas condicións que se terían nun entorno real en ARM de 64 bits. Resultou especialmente útil para comprobar o correcto funcionamento dos programas en arquitecturas ARM cando non se dispoñía de acceso á segunda plataforma hardware.

3.1.3. Docker

Esta ferramenta de creación de contedores permite configurar rapidamente un entorno de traballo dotado de todas as ferramentas necesarias para compilar, executar e depurar os programas e scripts empregados. Isto permite replicar as condicións do proxecto en calquera dispositivo e reducir así posíbeis discrepancias nos resultados debidas a modificacións no entorno común.

Para este traballo emprégase un ficheiro Dockerfile baseado na imaxe Ubuntu 24.04, que permite construír unha imaxe Docker na que se instalan tanto os

paquetes comúns para ambas as arquitecturas como aqueles específicos para cada unha.

3.1.4. Librerías auxiliares para PCA

Para a elaboración do programa que implementa o algoritmo de PCA, co fin de evitar a implementación manual de toda a lóxica necesaria para este algoritmo empregáronse librerías dispoñíbeis en C que xa implementaban algúns pasos esenciais deste algoritmo. En concreto os pasos de cálculo de valores e vectores propios de matrices simétricas así como a posterior multiplicación xeral de matrices pódense realizar empregando as librerías `lapacke.h` e `cblas.h` (e `armpl.h` en ARM) respectivamente.

As funcións empregadas son `lapacke_ssyev`, `cblas_sgemm` e `cblas_hgemm` das librerías `lapacke.h`, `cblas.h` e `armpl.h` respectivamente.

Tamén se realizará unha reimplementación da función `lapacke_ssyev` para que opere con todos os tipos de datos estudados. Esta nova función recibirá o nome de `lapacke_hfsyev` e será detallada en maior profundidade na Sección 3.2.

3.1.5. Ferramentas adicionais

Ademais dos materiais empregados anteriormente tamén resultaron de utilidade as seguintes ferramentas:

- QEMU (Quick Emulator): emulador de código aberto que emprega tradución de binarios dinámica para emular o procesador dun ordenador. Foi empregado para poder executar os programas compilados para ARM de 64 bits desde o dispositivo x86, cando non se dispoñía de acceso ao equipo con arquitectura ARM.
- Valgrind: conxunto de ferramentas de código libre para axudar á depuración de memoria en programas. Empregouse para verificar que todos os programas empregados neste traballo non teñan fugas de memoria.
- Git: sistema de control de versións distribuído empregado para rastrexar e xestionar as modificacións realizadas no código do proxecto, ademais de facilitar a creación e administración de ramas no proceso de desenvolvemento.
- GitHub: plataforma web empregada para manter nun repositorio remoto con todos os códigos do traballo e para poder acceder aos programas en ambos dispositivos de xeito rápido, sinxelo e fiábel. Tamén será empregado para acceder ao código fonte de `lapack`.
- Visual Studio Code: editor de código lixeiro e potente que ofrece extensións que nos permiten traballar con todas as linguaxes deste proxecto.

- Overleaf: editor empregado para a elaboración desta memoria, que ademais posúe unha ampla documentación tanto para o propio editor como para L^AT_EX.
- Intel Software Development Environment (Intel SDE): emulador de procesadores Intel que permite executar e depurar software en distintas xeracións de procesadores, probando novas instrucións. Empregouse para probar a posibilidade de executar programas non soportados nos nosos dispositivos pero o seu uso para a análise final foi descartado polo extremadamente baixo rendemento.

3.2. Metodoloxía

Nesta sección realizarase unha descrición dos procedementos deseñados e implementados, e do seu funcionamento, para amosar como axudan a resolver o problema de partida, e poder posteriormente discutir os resultados e sacar conclusións.

3.2.1. Programas empregados

Para este traballo seleccionáronse 4 algoritmos: AXPY ($A * X$ Plus Y), DCT (Discrete Cosine Transform), DWT_1D (Discrete Wavelet Transform para unha dimensión) e PCA (Principal Component Analysis). Foron implementados orixinalmente en 4 programas separados en linguaxe C pero posteriormente pasaron a ser 5 programas sen aumentar o número de algoritmos. A decisión de ampliar o número de programas radica na avaliación inicial dos tempos de execución dos programas e na posibilidade de implementar cos tipos de datos a estudar a función `lapacke_ssyev`.

Todos os programas seguen a mesma estrutura xeral para unha execución con valor n :

1. **Recepción de parámetros** Xestiónanse todos os parámetros recibidos e compróbase que n sexa un valor válido, un valor enteiro positivo, e en caso contrario remata a execución do programa prematuramente.
2. **Execución para un tamaño reducido** Dada unha constante `N_SMALL` (valores entre 4 e 6), que temos definida, realizamos unha execución da función que implementa o algoritmo do programa con datos para ese valor de n . Permítenos comprobar que o algoritmo implementado funciona correctamente para un tamaño fixo pequeno e para asegurarnos de que o programa en xeral compilouse correctamente e devolve resultados adecuados. Esta parte do programa sempre imprime na consola os datos iniciais e

finais independentemente dos argumentos do programa. Ao rematar a execución libera toda a memoria reservada para que a execución con tamaño n non se vexa afectada.

3. **Inicialización para a execución con tamaño n** Inicialízanse todas as variábeis necesarias para traballar cun tamaño n e, se temos recibido o flag `-v`, imprimimos por pantalla os todos os datos iniciais en notación científica con 10 díxitos para a parte decimal. A impresión indicará explicitamente “Datos ejecucion” para extraer o datos na Sección 3.2.2
4. **Execución do programa e medición do tempo de CPU con clock** Empregamos dúas variables de tipo `clock_t` (start e end) para capturar o contador de “ticks” de CPU antes e despois da invocación da función que implementa o algoritmo que queremos medir. Ao restarlle ao valor de end o de start obtemos o número de ticks consumidos exclusivamente polo noso código (non inclúe esperas en E/S ou suspensións do proceso). Posteriormente convertemos os ticks a segundos facendo un *cast* a `double` desta diferenza e dividindo por `CLOCKS_PER_SEC`, que indica canto dura un segundo en ticks. O resultado (`cpu_time_used`) é entón o tempo de procesador, en segundos, consumido pola execución do algoritmo.
5. **Remate do programa** Imprímese o tempo de execución do programa indicando explicitamente “Tiempo de ejecucion” para poder extraer os datos na Sección 3.2.2. Tamén se imprimirá o último elemento do array (ou matriz) de resultados e, se temos recibido o flag `-v`, imprimimos por pantalla os resultados de execución completos en notación científica con 10 díxitos para a parte decimal. A impresión indicará explicitamente “Resultados ejecucion” para extraer o datos na Sección 3.2.2. Finalmente libérase a memoria e remata o programa.

Os datos empregados por todos os parámetros son números reais pseudoaleatorios entre 0 e 10, xerados a partir dunha semente recibida polo programa o xerada na execución. Os valores xéranse con `rand()`, escálanse a un rango uniforme entre 0 e 1 (dividindo os datos xerados por `RAND_MAX`, facendo un *cast* en ambos operandos para evitar a división enteira) e multiplícanse por 10 para ter un rango entre 0 e 10.

Na versión orixinal de cada programa estes datos xéranse e almacénanse directamente para *float*, é dicir, fan o *cast* a este tipo de dato e almacénano directamente na variábel correspondente. Nas versións que traballan con datos de precisión reducida os datos xéranse de xeito indirecto a diferenza do programa orixina, posto que teñen que xerarse en *float* e posteriormente convértese ao tipo de dato empregado segundo corresponda. Afondarase máis neste aspecto na Sección 3.3.

AXPY

Descrición xeral AXPY pertence a BLAS de Nivel 1, que contén operacións básicas con vectores. Pola súa sinxeleza e frecuencia de uso, AXPY é unha das primeiras funcións optimizadas en calquera implementación de BLAS (como OpenBLAS, Intel MKL o cuBLAS).

Este programa, ademais da súa finalidade principal, serve para probar que todo funciona correctamente e que non existen erros co entorno, co método de compilación ou co cálculo de resultados. Tal e como indica o seu nome este programa implementa o algoritmo `axpy` para dous arrays `x` e `y` de lonxitude `n`. O programa principal, é dicir, o programa cuxo tempo de execución vai ser medido; é a función chamada `axpy`. Esta consiste nun bucle de tamaño `n` que en cada iteración almacena no array `y`, o resultado de multiplicar un escalar `a` polo elemento `i`-ésimo do array `x` e sumarlle o elemento `i`-ésimo do array `y`.

Parámetros da función central

`n` : número de elementos nos arrays `x` e `y`.

`a` : escalar polo que se multiplica `x`.

`x` : array de entrada (lonxitude `n`).

`y` : array de entrada/saída (lonxitude `n`).

DCT

Descrición xeral Este programa implementa a Transformada Discreta do Cóseno (DCT-II) unidimensional sobre un array `input` de lonxitude `n`. Non se trata da implementación real da DCT-II, é unha versión optimizada por recorrencia trigonométrica para realizar un cálculo máis eficiente dos resultados. O programa principal, é dicir, o programa cuxo tempo de execución vai ser medido; consiste na función `dct`. Esta función é unha versión da DCT-II que, en esencia, segue sendo unha DCT-II tradicional pero, en troques de invocar `cosf()` en cada paso do bucle interior, precomputa no bucle exterior o delta angular $\delta = \frac{\pi k}{n}$ e o theta angular $\theta = \frac{\pi k}{2n}$ para logo actualizar o coseno (e o seno) de forma recursiva empregando

$$\cos(\theta + \delta) = \cos(\theta) \cos(\delta) - \sin(\theta) \sin(\delta).$$

Deste xeito pasamos de $O(n)$ chamadas a $\frac{\cos f()}{\sin f()}$ por cada frecuencia `k` a tan só dúas chamadas (unha para `cos_delta` e outra para `sin_delta`) máis unhas multiplicacións sucesivas, acelerando significativamente o cálculo.

Parámetros da función central

input : array de entrada (const) de lonxitude n .

output : array de saída (lonxitude n) onde se gardan os coeficientes.

n_size : número de elementos en **input** e **output**.

DWT_1D

Descrición xeral Este programa implementa un único nivel da Transformada Wavelet Discreta unidimensional (DWT_1D) sobre un array **input_vector** de tamaño n . O noso programa realizará dúas execucións da función principal **convolve1d_generic**, partindo dos mesmos datos iniciais, un coa wavelet Le-Gall 5/3 (sen perda) e o outro coa wavelet CDF 9/7 (con perda). Isto ocasiona que, a diferenza de calquera outro programa dos empregados neste traballo, este programa teña por tanto dous tempos de execución e dous resultados de execución.

Posto que se trata só da Transformada Wavelet Discreta e non se fai a inversa, ao final da execución da función, **input_vector**[0.. $n/2-1$] contén coeficientes de aproximación e **input_vector**[$n/2$.. $n-1$] os de detalle.

Este deseño permite illar e medir cada etapa de filtrado e decimación nun só nivel. Se fose necesaria unha DWT multiescala completa, bastaría inserir esta función nun bucle de niveis que reaplique o filtro ao sub-vector de aproximación, xestionando tamén o padding nos bordes.

Parámetros da función central

input_vector : array de entrada/saída (tamén sae modificado).

vector_size : lonxitude do array.

kernels : filtros low-pass e high-pass da wavelet.

Datos do programa Os *kernels*, son datos de tipo **WaveletKernels**, un tipo de dato definido no programa que consta de catro campos:

low_pass_kernel : Punteiro a un array de **double** que representa o filtro *low-pass* da wavelet.

low_pass_size : Número de elementos no filtro pasa-baixo.

high_pass_kernel : Punteiro a un array de **double** que representa o filtro *high-pass* da wavelet.

high_pass_size : Número de elementos no filtro pasa-alto.

O seu valor asígnase mediante a función `initialize_kernels`, que reserva memoria e inicializa os catro campos cos valores correspondentes segundo a wavelet desexada, determinada por un parámetro de entrada.

PCA

Descrición xeral Este programa implementa o algoritmo de Principal Component Analysis (PCA) sobre unha matriz `matrix` de tamaño $n \cdot n$. O programa principal, cuxo tempo de execución vai ser medido, consiste na función `do_pca`, que implementa o algoritmo de pca paso a paso. A función realmente pode operar con matrices $n \cdot m$, pero para maior simplicidade á hora de introducir os datos e probar o funcionamento o programa previo á función traballa só para o caso $m = n$.

A función auxiliar `calculate_eigenvalues_and_eigenvectors` que calcula os vectores e valores propios mediante a función `lapacke_ssyeve` e ordénalos mediante o algoritmo quicksort, correspondente aos paso 3 e 4 emprega.

Para o paso 5 emprega a función auxiliar `transform_data` que realiza a transformación de los datos (“proxección dos datos”) mediante a función `cblas_sgemm`.

Parámetros da función central

`matrix` : matriz de entrada/saída que contén os datos sobre os que se realizarán as operacións.

Datos do programa Todas as matrices coas que se traballan neste programa son de tipo `Matrix`, un tipo de dato definido no propio código que consta de tres campos:

- `rows` Número de filas.
- `cols` Número de columnas.
- `data` Punteiro a punteiros que representa os datos como un array bidimensional (array de arrays).

Diferenzas relevantes entre versións Estas versión que non empregan *float* presentan un problema derivado precisamente dos tipos de datos empregados, as funcións que se empregan na versión orixinal só están deseñadas para traballar con tipos de datos *float*. Este problema resólvese de dous xeitos distintos en función da arquitectura na que se realiza a execución e xestiónase mediante compilación condicional, para poder empregar un só ficheiro en distintas arquitecturas.

- x86: Non hai ningún substituto para as funcións externas `lapacke_ssyeve` e `cblas_sgemm`. En todos os programas que non traballan con *float* emprégase

un buffer intermedio de *float* para executar as chamadas á función, ao rematar cópanse ps datos e libérase a memoria asignada a ese buffer.

- ARM: procedemento igual a x86 para a función `lapacke_ssyev`. Respecto á función `cblas_sgemv`, a librería `armpl.h` ofrece a función `cblas_hgemv`, que traballa con `__fp16`. Ao poder empregarse esta función os programas que traballan con `_Float16` e `__bf16` empregarán un buffer intermedio de `__fp16`, en troques de *float*, para executar `cblas_hgemv`.

Afondarase máis nestas diferenzas na Sección 3.3.

PCA_REIMPL

Descrición xeral Este programa implementa o algoritmo de Principal Component Analysis (PCA) sobre unha matriz `matrix` de tamaño $n \cdot n$. Trátase dunha modificación do programa PCA descrito na Sección 3.2.1 para as versións que empregan datos en punto flotante de precisión reducida, é dicir, as versións *FP16*, *FP16_ARM* e *BF16*.

A diferenza chave respecto a PCA radica en que agora estas versións empregan a función `lapacke_hfsyev` e se deixa de usar `lapacke_ssyev`.

Todo o demais é exactamente igual que ao descrito na Sección 3.2.1 excepto polo feito de que agora xa non é preciso empregar un buffer intermedio, para calcular os vectores e valores propios, cando non se traballe con *float*, posto que empregamos a función `lapacke_hfsyev`.

Función `lapacke_hfsyev` Reimplementación da función `lapacke_ssyev` para traballar con datos do tipo `lapack_float`, un tipo definido no programa que pode representar `__fp16`, `__bf16` ou `_Float16` (por defecto). Deste xeito, a función pode operar de maneira agnóstica ao tipo de dato subxacente, permitindo escoller o formato adecuado en tempo de compilación mediante directivas.

O tipo de dato escóllese en tempo de compilación en función das macros `USE_FP16`, `USE_BF16` ou ningunha delas, para os tipos de datos `__fp16`, `__bf16` ou `_Float16` respectivamente. Recoméndase empregar a macro `USE_Float16` para garantir homoxeneidade e claridade no proceso de compilación para `_Float16` aínda que non é obrigatorio.

Esta reimplementación é completa e autocontida: reescribe todos os subprogramas que poderían ser invocados durante a execución da función, pero unicamente eses, e tamén todas as macros necesarias para que funcione correctamente.

Isto implica implementar os subprogramas en linguaxe C empregando o tipo `lapack_float` no canto de `float`, así como traducir a C os subprogramas orixinais escritos en Fortran e adaptalos ao novo tipo de dato. As funcións non mudan o seu funcionamento interno (excepto casos puntuais) nin os algoritmos que empregan durante a execución. Para cumprir con isto reimplementáronse en

linguaxe C un total de 49 funcións Fortran e modificáronse un total de 9 funcións en linguaxe C.

Todas as funcións renomeadas e modificadas teñen un novo nome (a excepción de `lapacke_xerbla`) e se caracterizan porque o prefixo “s” que indicaba *float* (*single-precision*) foi substituído por “hf”. O principal motivo polo que se ten escollido este prefixo é para que sexa indicativo de que se trata de *half-precision* (porque os datos ocupan a metade que *single precision*) e para evitar posíbeis problemas de nomes se nun futuro son implementadas oficialmente. Con isto todas as funcións específicas de *float* como `ssterf` ou `LAPACKE_sge_trans`, pasan a ter este prefixo (serían `hfsterf` e `LAPACKE_hfge_trans`). Nalgúns programas que realmente o uso de `lapack_float` non resulta relevante (ou non teñen o prefixo “s”) engádense os sufixos `_reimpl` ou `_half_precision` (ou ambos). Estes prefixos indican que é reimplementada de Fortran ou que foi modificada para traballar con `lapack_float` respectivamente.

Entre estas funcións destaca a función `lsame_reimpl`, que foi adaptada durante a reimplementación para ser só válida en sistemas ASCII. Este tipo de reimplementación tamén recibiu a función `ilaenv_reimpl_half_precision`, pero mantendo exactamente o mesmo comportamento, o que implica que as invocacións en determinadas funcións teñen que facerse co nome anterior.

Tamén foi necesario crear novas macros para os datos de *binary16* e *bfloat16* para que funcións coma `hflamch_half_precision` proporcionen datos acordes ao tipo de dato que envolve *lapack_float*. Adicionalmente creáronse funcións para o cálculo da raíz cadrada e o valor absoluto, declaradas en `definitions.h`, para posibilitar a operación con este tipo de dato sen ter que empregar buffers ou variábeis intermedias. No caso da raíz cadrada saíase do alcance do traballo así que os casos especiais están implementados e a propia raíz depende da función `sqrthf`.

Cómpre destacar que estas reimplementacións, ao tratarse simplemente de traducións de linguaxe ou adaptacións tipadas, seguen estando suxeitas á licenza orixinal de LAPACK.

Durante o desenvolvemento desta reimplementación atopámonos con algunhas dificultades que serán tratadas na Sección 3.3.

Outros programas

No directorio Tests-Python, localizado no directorio raíz do proxecto, están os programas chamados `axpy_test.py`, `dct_test.py`, `dwt_1d_test.py` e `pca_test.py` e son empregados para comprobar os resultados dos programas anteriores. Para usalos se dispón do script `setup-python-env.sh` que crea un entorno Python en que poder executalos.

3.2.2. Scripts empregados

Nesta sección detallaranse todos os scripts que foron empregados tanto durante o desenvolvemento e depuración dos programas como durante a obtención dos datos e a creación de métricas.

Compilación

Todos os directorios que almacenan cada grupo dos programas anteriores contan con 3 scripts de compilación no directorio no que se atopan, un por caso contemplado, e encárganse de compilar todos os programas para *Intel*, *AMD* ou *ARM*.

Para este traballo decidiuse empregar a seguinte convención para os executábeis xerados ao compilar:

- Nome: o nome do executable resultante da compilación será o mesmo nome que o ficheiro orixinal.
- Extensión: todos os programas compilados se son para arquitectura x86 (independentemente de que sexan en Intel ou en AMD) non terán extensión e se son para ARM (compilación nativa ou cruzada) empregarán “.out” como extensión.

Os scripts compilan todos os programas do directorio con `gcc-14` e os flags `-O3` e `-fomit-frame-pointer`, ademais das opcións de ligazón necesarias para as librerías, para optimizar o rendemento e liberar un rexistro extra. Deste xeito garantimos igualdade entre arquitecturas, pois `-O3` non sempre activa por defecto `-fomit-frame-pointer`.

Como xa se mencionou na función `lapacke_hfsyev`, `pca_reimpl` compílase adicionalmente cos flags `-DUSE_FP16`, `-DUSE_BF16` ou `-DUSE_Float16` segundo o programa empregue `__bf16`, `__fp16` ou `_Float16`.

Tamén existen diferenzas substanciais entre os scripts para ARM e os demais posto que, como xa se ten comentado no Capítulo 2 o soporte por arquitecturas varía.

- Scripts Intel: Compílanse cos flags: `-march=tigerlake -mtune=tigerlake -mf16c`. Antes de xerar cada executable compróbase en tempo de compilación se hai soporte para AVX512FP16 e/ou AVX512BF16; se non, só se crea o executable sen soporte (segundo a convención anterior).

Cando hai soporte, xéranse tres variantes adicionais para cada programa que emprega `_Float16` e `__bf16`. Cada sufixo no nome dos executábeis indica o uso dos seguintes flags incrementais:

`_native-base: -mavx512fp16` ou `-mavx512bf16`

```
_avx512_precision: + -fexcess-precision=16
_max_performance: + -mfpmath=sse
```

- Scripts AMD: a nivel de funcionamento é igual ao de Intel pero non emprega os flags `-march=tigerlake` e `-mtune=tigerlake` porque non é un procesador Intel e non se dispón dun procesador no que probalo. Simplemente quedou cun formato que engade os flags `-march=znver6` e `-mtune=znver6` cando se compila para `_Float16`.
- Scripts ARM: ao igual que nos anteriores emprega o flags `-O3`, e adicionalmente emprega `-ftree-vectorize`, aínda que xa está contemplado no anterior. Tamén emprega o flag `-march=armv8.2-a+fp16+fp16fml+simd` que indica a arquitectura do procesador, mellora o procesamento en paralelo e engade soporte para uso de instrucións con datos *binary16*, é dicir, `__fp16` e `_Float16`.

Adicionalmente existe o script de compilación xeral chamado `compile_all.sh` para compilar todos os programas.

Execución

Todos os directorios que almacenan cada grupo de programas dos algoritmos contan con 3 scripts de execución no directorio no que se atopan, un por caso contemplado, e encárganse de executar todos os programas para *Intel*, *AMD* ou *ARM*.

Estes scripts de execución consisten nun bucle que percorre todos os ficheiros executábeis e se cumpren un criterio execútaos cos argumentos recibidos.

As únicas diferenzas entre os scripts en ARM e en Intel/AMD son:

1. Criterios de selección de ficheiros

ARM : só executábeis con extensión `.out`.

Intel/AMD : todos os executábeis sen extensión (sen `.sh`, `.out` ou `.o`).

2. Execución emulada (criterios invertidos para os ficheiros: Intel/AMD filtran como ARM e viceversa)

ARM : emúlase con Intel SDE (`-spr`).

Intel/AMD : emúlase con QEMU.

Adicionalmente existe o script de execución xeral chamado `run_all.sh` para executar todos os programas compilados.

Obtención dos datos

Para a obtención tanto do tempo de execución coma dos datos (datos iniciais e resultados) da execución empréganse dous scripts: `tiempo_ejecucion.py` e `resultado_ejecucion.py`. Para executalos poderíase empregar o entorno configurado polo script `setup-python-env.sh` se non se dispón dos paquetes que se precisan instalados no equipo. Empregar este entorno é o método recomendábel para executar os scripts se se está a facer mediante Docker.

Para obter o tempos de execución emprégase o ficheiro `tiempo_ejecucion.py`, que permite obter os tempos de execución dun programa recibido como argumento.

O ficheiro `resultado_ejecucion.py` permite obter os resultados de execución e os datos antes das operacións se recibe o flag “-q”.

Tamén contamos con scripts en bash que xestionan a extracción destes datos para todos os programas o que permite automatizar a extracción destes valores sen ter que executar para cada programa. En concreto o script `get_all_times.sh` permitiría extraer todos os tempos de execución, `get_all_results.sh` faría o mesmo para os resultados de execución.

- `get_all_times.sh`: Permite extraer todos os tempos de execución de todos os programas. Executa o programa `compile_all.sh` e posteriormente percorre unha lista de directorios dos que obtén os programas, seguindo os mesmos criterios que os scripts de execución. Finalmente pásalos un por un a `tiempo_ejecucion.py` indicándolle que faga 13 execucións para todos, excepto para DCT que serán 7.
- `get_all_results.sh`: Permite extraer os datos de execución de todos os programas. Executa o programa `compile_all.sh` e posteriormente percorre unha lista de directorios dos que obtén os programas, seguindo os mesmos criterios que os scripts de execución. Finalmente pásalos un por un a `resultado_ejecucion.py` indicándolle que o valor de n será 15000 excepto para os directorios PCA e PCA_REIMPL que será 1000 e que ten que recoller tanto os datos iniciais como o resultado final.

3.3. Retos durante o desenvolvemento

Durante a elaboración dos programas atopámonos cunha serie de retos que serán detallados a continuación xunto coa solución á que se chegou para resolvelos.

1. Xeración de datos aleatorios: o método empregado para a xeración de datos aleatorios ocasionaba que ao ser implementada directamente para tipos de datos de precisión reducida os datos xerados sempre fosen NaN. Para solucionalo optouse por xerar todos os valores en formato *float* e posteriormente faise un *cast* ao tipo de dato obxectivo.

2. Soporte para AVX512FP16 e AVX512BF16: co procesador empregado para probar os programas en arquitectura x86 non dispón de soporte para instrución AVX512FP16 nin AVX512BF16 barallouse a posibilidade de empregar Intel SDE pero foi desbotada debido ao pobre rendemento amosado.
3. Implementación nunca nativa de PCA: para implementar o algoritmo de pca recorreremos ás función `lapacke_ssyeve` e `cblas_sgemm` que non teñen versións cos datos de estudo, agás `cblas_hgemm` que non é completamente válida porque só traballa con `__fp16` e en ARM. Para solucionalo empréganse buffers intermedios antes de invocar estas funcións. Posteriormente faise tamén unha reimplementación de `lapacke_ssyeve` permitindo así ter unha implementación máis próxima a unha completamente nativa ao quedar só a función `cblas_sgemm` que ademais conta cunha versión alternativa en ARM.
4. Tempo de execución de DCT moi elevado: o tempo de execución do algoritmo de DCT-II orixinal crece exponencialmente. Para solucionalo reduciuse a lista de valores de n empregada para medir os tempos ata só $n = 1742848$ pero foi insuficiente. Adicionalmente modificouse a función que implementaba o algoritmo de dct para que o fixese mediante unha recorrencia trigonométrica para reducir a súa complexidade. Isto segue sen unha solución idónea e os tempos de execución son moi elevados.
5. Falta de datos para análise rigorosa de DCT: como o tempo de execución do algoritmo DCT-II é tan elevado incluso despois das optimizacións efectuadas non se dispón dos tempos de execución para as execucións na arquitectura ARM, nin para a do tipo de dato `__bf16` na arquitectura x86.
6. A función `lapacke_hfsyev` ten unha gran perda de precisión: derivado de facer todas as operacións en *half precision float* ou en *bfloat16* os resultados desvíanse moito dos esperados. Para solucionar isto probouse a implementar algunhas funcións chave internamente con *float* para mellorar a precisión pero impacto foi nulo así que só se deixaron implementadas con este tipo de dato as imprescindíbeis.
7. Macros de GCC con valores incorrectos: as macros `__FLT16_MAX_10_EXP__` e `__FLT16_MIN_10_EXP__`, así como as macros `__BFLT16_MAX_10_EXP__` e `__BFLT16_MIN_10_EXP__` conteñen uns valores non acordes cos datos reais. Para solucionalo os valores foron introducidos manualmente en cumprimento co estándar.
8. Erro na función `lapacke_hfsyev` para $n > 129$: cando a matriz ten unhas dimensións de $130 \cdot 130$ ou superiores realízanse escrituras e lecturas fora dos límites debido ao valor asignado á variábel `lwork`. Para solucionar isto cando

$n > 129$ asignámoslle a `lwork` o valor $3*n-1$, que é o valor mínimo admitido por `lapacke_hfsyev_work`, reimplementación de `lapacke_ssyev_work`.

9. Erro de reserva de memoria na función `lapacke_hfsyev`: cando a matriz ten unhas dimensións superiores a $2000 \cdot 2000$ o cálculo orixinal de `lwork` ocasiona que `lwork` teña valor overflow e por tanto salte un erro ao non poder reservar esa memoria. Foi resolto coa solución ao problema anterior.

3.4. Métricas de rendimento

Para a poder realizar a avaliación do rendimento en función dos datos obtidos previamente empregáranse unhas métricas de rendimento, que están diferenciadas entre métricas para tempos de execución e métricas para valores de execución, tanto para resultados como para datos iniciais.

3.4.1. Métricas para tempos

Para calcular o rendimento relativo aos tempos de execución para o uso de tipos de datos en punto flotante de precisión reducida a métrica que se empregará é o **speedup**. Calculárase como o tempo de execución dunha versión do programa respecto á que traballa con *float*. Permitirá ver como varía o rendimento ao traballar cuns tipos de datos ou outros e tamén como é esta diferenza de rendimento en función das operacións que se realizan.

3.4.2. Métricas para resultados

Para avaliar o rendimento no referido á precisión dos tipos de datos empregáranse as seguintes métricas: **Erro Absoluto Medio (MAE)**, **Erro Cadrático Medio (MSE)**, **Raíz do Erro Cadrático Medio Normalizada (NRMSE)** e **Coefficiente de correlación de Pearson**. Cabe destacar que non todas as métricas serán aplicadas a todos os algoritmos posto que, debido á natureza dos mesmos, non todas son completamente aplicábeis para todos os algoritmos.

Todas as métricas calcularanse en base ao resultado das operacións para os tipos de datos de precisión reducida respecto aos valores obtidos dos programas que empregan *float* por considerarse os valores correctos.

Erro Absoluto Medio (MAE)

O MAE calcula a media das diferenzas absolutas entre os valores orixinais e os valores resultantes tras aplicar o algoritmo cos distintos tipos de datos. Esta métrica ofrece unha lectura directa da desviación media, sen amplificar de forma excesiva os erros maiores, polo que resulta moi útil en casos sinxelos como AXPY, onde o patrón de erro é homoxéneo e non queremos penalizar outliers. En procesos

máis complexos como DCT, DWT_1D e PCA, o MAE permite cuantificar de forma transparente a perda de precisión media, servindo de complemento ao MSE para detectar erros recorrentes que non aparecen amplificados polos cadrados.

Erro Cadrático Medio (MSE)

O MSE mide a media das diferenzas elevadas ao cadrado entre os valores orixinais e os valores procesados cos outros tipos de datos, destacando erros grandes debido ao efecto de ponderar as discrepancias ao cadrado. Esta característica fai que o MSE sexa moi sensible a outliers, o que en algoritmos como DCT, DWT_1D e PCA é esencial para garantir que ningún pico de erro pase desapercibido. En AXPY, a pesar da sinxeleza do cálculo aritmético, o MSE segue a ser unha referencia valiosa para cuantificar a precisión absoluta en termos enerxéticos. Aplicar o MSE ofrece unha visión complementaria ao MAE, facendo fincapé nas maiores desviacións que poderían corromper a saída.

Raíz do Erro Cadrático Medio Normalizada (NRMSE)

A NRMSE obténse ao calcular o RMSE (raíz cadrática do MSE) e logo normalizar este valor polo rango ou media dos datos orixinais, ofrecendo unha métrica adimensional que facilita a comparación entre algoritmos e distintas escalas de datos. Esta normalización é clave cando se queren comparar resultados en algoritmos heteroxéneos como DCT, DWT_1D e PCA, ou incluso en operacións máis sinxelas como AXPY, xa que compensa as diferenzas de magnitude intrínsecas dos sinais. Permite avaliar de forma equitativa o impacto da precisión numérica na calidade global do proceso, independentemente da escala. Esta métrica resulta especialmente valiosa ao combinar resultados de múltiples contextos pouco relacionados, proporcionando unha base unificada de comparación. Os valores desta métrica tamén poden expresarse en porcentaxe ao multiplicalo por 100.

Coefficiente de Correlación de Pearson

O coeficiente de Pearson mide a relación lineal dos resultados. Ofrece un valor entre -1 e 1 que reflicte se os patróns de variación se manteñen, sen cuantificar directamente a magnitude do erro. Esta métrica é esencial en transformacións complexas como PCA, onde conservar a estrutura de variación interna garante a fidelidade do resultado. Mesmo en AXPY, o coeficiente de Pearson serve para validar se o cambio de tipo de dato impacta na coherencia global dos resultados. Ao aplicalo, obtense unha perspectiva estatística que complementa as métricas de erro, enfocándose na integridade do patrón lineal da saída.

Capítulo 4

Probas e discusión dos resultados

Neste capítulo preséntanse de xeito conciso as probas realizadas e a súa análise crítica para validar a funcionalidade, o rendemento e a escalabilidade da solución desenvolvida. A continuación, amosaranse táboas e gráficas cos datos obtidos, interpretándoos cuantitativamente e contrastándoos coas expectativas iniciais.

4.1. Discusión dos resultados

Nesta sección presentaremos os datos obtidos dos programas anteriores indicando posíbeis conclusións que se poden sacar dos mesmos, para isto primeiro centrarémonos nas métricas relativas ao desempeño e posteriormente nas relativas á precisión dos datos.

4.1.1. Métricas de desempeño

Tras a execución dos programas que implementan os algoritmos atopámonos cunhas métricas bastante diferenciadas entre arquitecturas e entre programas. Por este motivo primeiro farase unha análise por algoritmos, posteriormente unha breve comparativa e finalmente unha análise da situación global. Cabe destacar o feito de que debido a dispoñibilidade do sistema co procesador x86, existe unha mostra máis ampla de valores de n que para ARM, pero esta mostra é unha ampliación con respecto á de ARM, polo que os primeiros 19 valores son os mesmos en ambos casos.

Algoritmo AXPY

En programas sinxelos como o algoritmo axpy, para tamaños moi pequenos de n cando os datos caben en caché atopámonos con dúas situacións. Por un lado, cando se traballa con datos que non poden realizar a operación ben por non ter soporte, como no caso de x86, ou ben por ser datos de almacenamento, como `__fp16`; as conversións en cada operación teñen un gran impacto no rendemento,

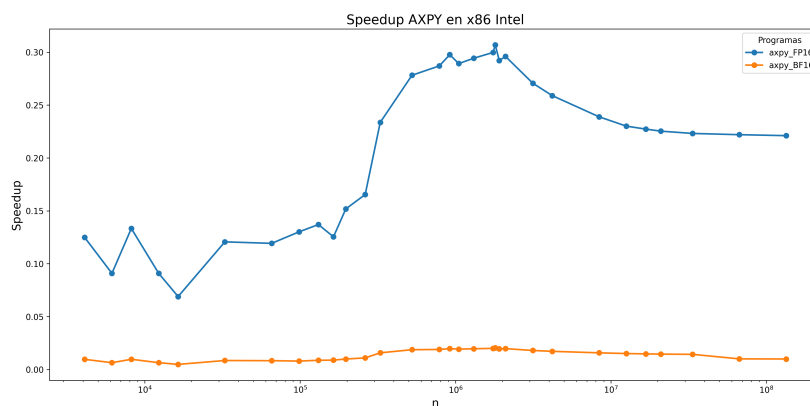


Figura 4.1: Speedup acadado no algoritmo axpy respecto da versión de 32 bits na arquitectura x86 cun procesador Intel.

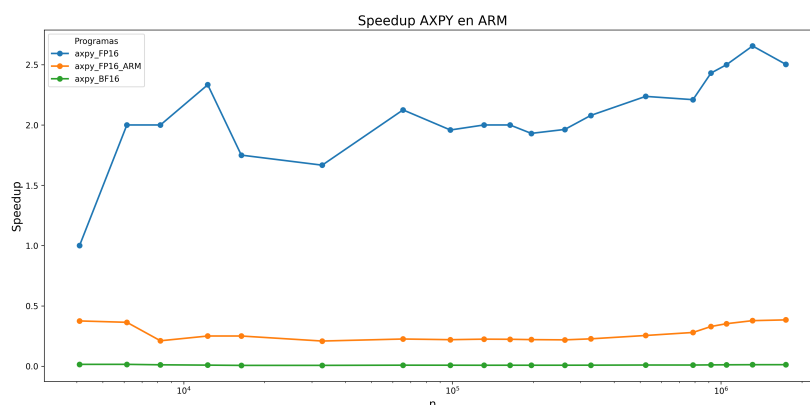


Figura 4.2: Speedup acadado no algoritmo axpy respecto da versión de 32 bits na arquitectura ARM.

facendo que baixe o speedup. Esta baixada é máis pronunciada nos datos de só almacenamento, segundo o observado na Figura 4.2, que cando simplemente non hai soporte, como podemos apreciar na Figura 4.1, posibelmente porque as instrucións `f16c` axudan a moderar esta caída. Por outro lado en ARM que hai soporte, cando os datos non son de almacenamento, a situación é a inversa cun aumento do rendemento de ata un 2,3% para os valores nos que o programa aínda cabe en caché L1 e a versión base xa non dispón desta memoria tan rápida. Conforme crecen aínda máis os valores de n a evolución é máis uniforme para todos os casos, cando a versión base baixa outro nivel da memoria caché os demais programas aínda desfrutan de memoria de maior velocidade. É neste momento cando o rendemento volve subir destacando especialmente no caso de x86 no que esta diferenza ocasiona que o overhead das conversións sexa menor que a diferenza de velocidade das memorias e o speedup case se duplique. En ARM tamén hai un aumento que eleva os rendementos ata o máximo valor acadado

pero, este crecemento é apenas notábel cando o dato é de almacenamento, e na execución nativa non resulta tan grande porque xa tivera un aumento non presente nos outros casos. Finalmente en x86 pódese observar como cando a versión FP16 e o programa base están ambos en RAM a vantaxe de rendemento decrece pero sendo maior que a inicial. Con todo, esta versión de x86 (que non conta con soporte nativo para as operacións) e a FP16_ARM, que emprega o tipo de dato que é só de almacenamento, como moito aspiran a ser 3 veces máis lentos que o programa base. A situación é incluso máis desfavorábel para *bfloat16* onde, pese a seguir as mesmas tendencias que *binary16*, o speedup máximo é de 0,020 facendo que o uso deste dato sexa contraindicado para este contexto, polo menos se non está soportado.

Algoritmo DCT

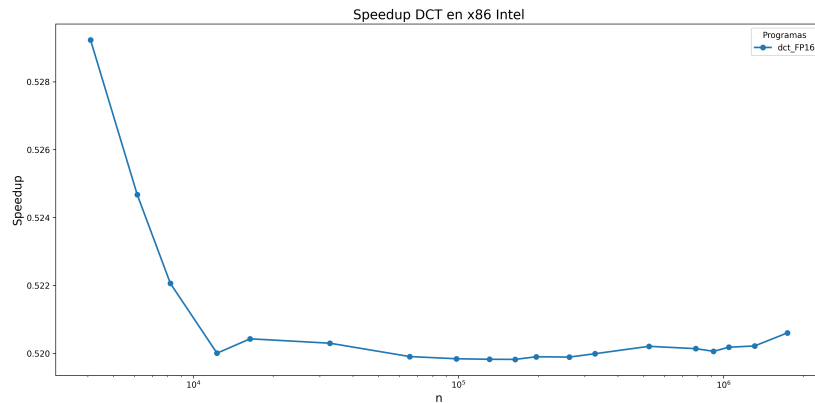


Figura 4.3: Speedup acadado no algoritmo dct respecto da versión de 32 bits na arquitectura x86 cun procesador Intel.

Programas máis complexos como o do algoritmo DCT que emprega operacións complexas como as trigonométricas atopámonos cunha situación moi diferente á do algoritmo axpy. Como se pode apreciar na Figura 4.3, cando se traballa con *binary16* o programa precisa sempre aproximadamente o dobre de tempo para realizar estas operación para calquera valor de n . Esta estabilidade pode deberse ao feito de que as operacións realízanse en *float*, é dicir, está nunha precisión mixta axudada polas instrucións **f16c**. A ausencia doutras versións e tipos de datos impide contextualizalo pero o rendemento respecto ao programa orixinal permítenos concluír que non é recomendábel usar *binary16* para este contexto.

Algoritmo DWT_1D

Programas como o do algoritmo DWT_1D, permítennos avaliar o rendemento dun mesmo programa para distintos datos “constantes”, é dicir, manter os datos

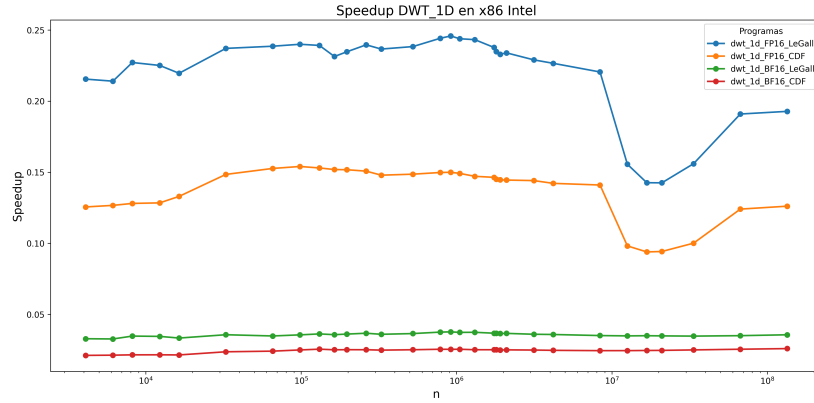


Figura 4.4: Speedup acadado no algoritmo dwt nunha dimensión respecto da versión de 32 bits na arquitectura x86 cun procesador Intel.

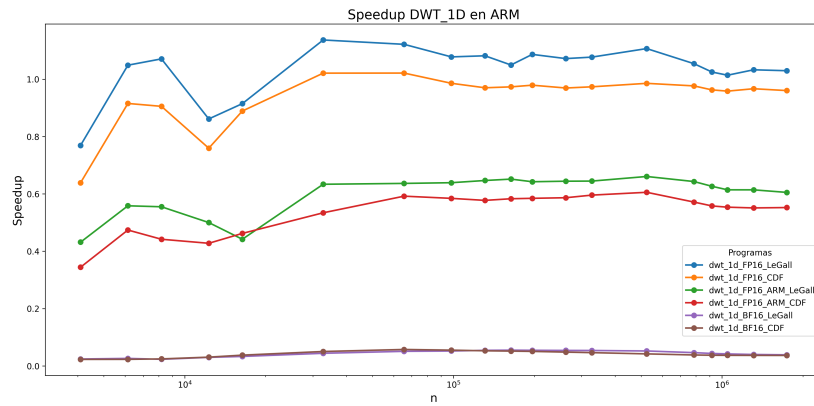


Figura 4.5: Speedup acadado no algoritmo dwt nunha dimensión respecto da versión de 32 bits na arquitectura ARM.

inicias pero variar algúns datos cos que se fan as operacións, neste caso os filtros das wavelets. Neste algoritmo destaca o feito de que o rendemento é lixeiramente inferior que no caso de axpy cando non hai soporte, como se pode apreciar na Figura 4.4, e moi inferior cando existe este soporte, como podemos ver na Figura 4.5. Contrariamente ao que sucede con axpy a versión FP16_ARM ten mellores datos, aspirando a ser 1,5 veces máis lento, indicando que nestas operacións pode verse máis favorecido. En xeral os speedups son bastante estábeis para todos os tipos de datos excepto para x86, onde destaca bastante unha caída do 30 % do rendemento, para valores de n moi elevados xa cando o programa precisa moita memoria e os datos están en ambos casos na RAM, mingando o de por si moi pobre rendemento. *bfloat16* está na mesma situación que axpy onde resulta completamente irrelevante pese a render lixeiramente mellor que nese algoritmo e ter en xeral unha tendencia ascendente contraria a *binary16*, aínda que mantendo a diferenza de rendemento entre filtros de wavelet. En liña do observado

anteriormente o uso de *bfloat16* e *binary16* sen soporte, ou mediante datos só de almacenamento, non é rendíbel e incluso o uso con soporte queda condicionado por ter un rendemento ás veces inferior ao caso base.

Algoritmo PCA

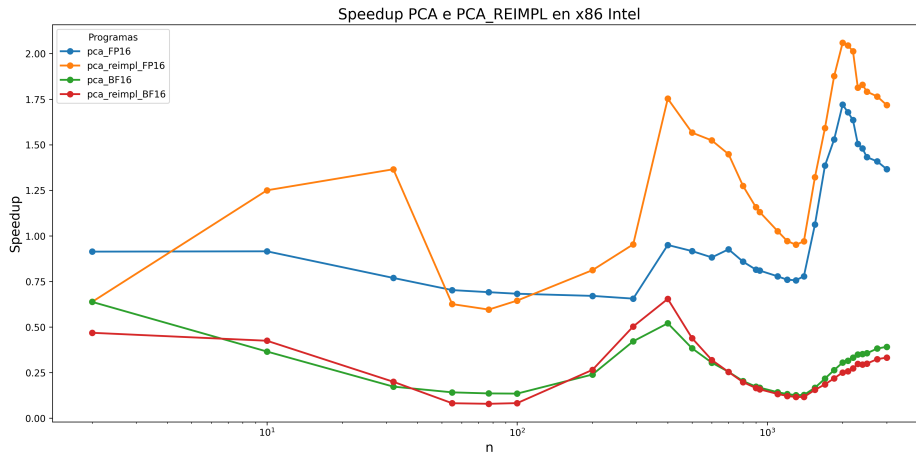


Figura 4.6: Speedup acadado no algoritmo *pca*, con e sen a reimplementación de *lapacke_ssyev*, respecto da versión de 32 bits na arquitectura x86 cun procesador Intel.

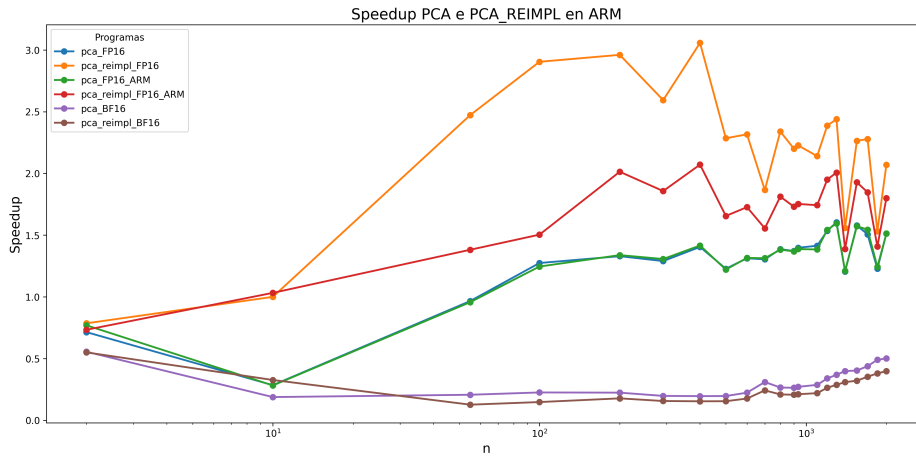


Figura 4.7: Speedup acadado no algoritmo *pca*, con e sen a reimplementación de *lapacke_ssyev*, respecto da versión de 32 bits na arquitectura ARM.

Programas moito máis complexos, como o que implementa o algoritmo de PCA a situación cambia completamente para mellor respecto aos algoritmos anteriores para todos os tipos de datos estudados, tal como se observa na Figura 4.6

e na Figura 4.7. Como a situación cambia entre x86, que non ten soporte nativo para *binary16*, e ARM, que si ten; para este algoritmo primeiro farase unha análise separada por arquitecturas antes de comparalas.

En arquitecturas x86 sen soporte, atopámonos cunha situación de lixeiro descenso para valores pequenos de n ata que cando o programa base chega á caché L3, o rendemento do programa FP16 increméntase ata situarse case á par do programa base. O overhead introducido polas conversións para operacións aritméticas (atenuado ao usar instrucións *f16c*) e o dos buffers intermedios ten un gran impacto cando a memoria é moi rápida. Nos valores medianos a situación repítese porque o número de operacións e o tamaño dos buffers aumenta moito e cada vez precisa máis memoria o programa. Con valores grandes de n , cando xa non cabe ningún dos programas en caché, o rendemento aumenta ata máis do dobre chegando a ser ata un 72 % máis rápido no mellor dos casos e moderándose ata un 40 % para o valor de n máis grande. Este comportamento é lóxico, cando os programas compiten en igualdade de condicións no relativo á velocidade da memoria o overhead ten un impacto determinante no rendemento, en desigualdade a velocidade da memoria é determinante. Son as diferenzas de ancho de banda e a maior dispoñibilidade, proporcionalmente, de memoria de maior velocidade o que ocasiona que, en xeral, o speedup aumente co valor de n .

No programa de x86 que tamén implementa PCA coa reimplementación de *lapacke_ssyev* o comportamento é similar ao anterior excepto por dous factores: o rendemento xeral é superior ao da versión anterior e para valores moi pequenos existe un aumento que antes non había. O incremento que se dá nos valores pequenos obedece a que agora empréganse a menos buffers intermedios, é dicir, menos memoria, pese a haber máis operacións aritméticas (cun impacto atenuado polo uso de instrucións *f16c*). Estes factores ocasionan o mencionado aumento cando o programa FP16 ocupa só a caché L1, e logo son a causa principal do mellor rendemento xeral, e a acentuación dos aumentos de rendemento, respecto á versión que emprega *lapacke_ssyev*. Con todo, existe a posibilidade de que parte da optimización estea motivada pola implementación en linguaxe C dos programas en Fortran que se invocarían na función orixinal, pero como queda fora do alcance deste traballo non se estudará.

Na arquitectura ARM, con soporte para operacións con *binary16*, a situación xeral é diferente destacando especialmente o feito de que o rendemento para *_Float16* e para *__fp16* é exactamente igual, quitando a marxe de erro, nos programas que empregan *lapacke_ssyev*. Este feito é debido moi probabelmente a que ambos tipos de datos empregan un buffer intermedio para esta función, que é a que maior impacto ten no tempo de execución, sumado a que a versión FP16_ARM non precisa dun buffer intermedio para *cblas_hgemv* e en conxunto ocasiona esta situación. Respecto ás tendencias no rendemento obsérvase que todos os programas presentan un crecemento do rendemento desde o valor inicial ata valores medianos de n onde xa o programa chega á memoria RAM. A partir de aquí, para valores medianos, o rendemento causa un lixeiro crecemento ao usar

`lapacke_ssyev` mentres que o uso da reimplementación ocasiona un decrecemento. O crecemento estábel cando se usa `lapacke_ssyev` débese ao feito, observado anteriormente, de que o overhead do buffer intermedio ten cada vez menor impacto fronte ás operacións do programa. Pola contra, a baixada de rendemento cando se usa a versión coa reimplementación radica en que xa se superou a zona onde se beneficiaba simultaneamente da maior velocidade da memoria, maior ancho de banda, e da ausencia de overhead polo uso de buffers intermedios. Como no límite dos valores de n de tamaño medio acada un tempo de execución 3 veces menor que o programa base, ao non dispoñer de tantos factores favorábeis o tempo de execución pasa a ser só da metade, sigue sendo unha gran mellora pero menor en comparación. Para valores grandes de n , $n = 1200$, todos os programas volven ter o mesmo comportamento: unha tendencia descendente cunha oscilación do speedup. Esta oscilación é entre 2,4 e 1,5 para a versión coa reimplementación e entre 1,6 e 1,2 para a sen reimplementación. A tendencia oscilante pode estar causada polo feito de que nalgúns puntos existan máis fallos caché ou saturación do ancho de banda nalgúns operacións, respecto á versión base, ocasionando puntos concretos de caída drástica. A causa do decrecemento ven principalmente da maior cantidade de datos en memoria RAM, que reduce o beneficio que aporta a memoria caché, sumado ao impacto crecente do buffer intermedio, só para a versión FP16, e o overhead das operacións aritméticas crecentes, só para a versión FP16_ARM.

Para os programa que empregan *bfloat16*, a situación non mudou en absoluto e o rendemento segue sendo moi pobre pese a que no mellor caso chegaría a ser só un 30 % máis lento que o caso base, que é moito máis rápido que en calquera algoritmo anterior pero dista de ser sequera aceptábel.

En liñas xerais, todos os tipos de datos *binary16* teñen un mellor rendemento na versión que emprega a reimplementación da función `lapacke_ssyev`, motivado polo menor uso de buffers intermedios que teñen un impacto maior no rendemento que o overhead das operacións aritméticas (inexistente se hai soporte). O único tipo de dato que ten un peor rendemento nesa versión é `__bf16` que, en xeral, sitúase constantemente cun rendemento máis pobre. Esta baixada do rendemento ven dun aumento das operacións aritméticas con este tipo de dato e podería ser indicativo de que nun procesador con soporte para este tipo de dato, por tanto sen overhead nas operacións, esta implementación podería ser máis eficiente.

Comparativa entre arquitecturas

O que máis se pode apreciar ao observar ambas arquitectura é que dispoñer de instrucións que podan traballar cos datos de estudo é o factor máis relevante. De feito se comparamos as gráficas de AXPY Figura 4.2 e Figura 4.1 aprécianse claramente as melloras nun contexto no que as operacións aritméticas teñen a maior parte do peso do programa. Esta diferenza é en media dunha orde de magnitude pero no caso máis extremos chega a ser de dúas ordes de magnitude

o que indica claramente.

Cabe destacar que, para a situación representada pola Figura 4.6 podería semellar que se alcanza un speedup maior que na arquitectura ARM o certo é que a Figura 4.7 ten unha mostra menor. O que implica que realmente o único punto no que o speedup da arquitectura x86 supera ao de ARM é para $n = 2000$ e non se sabe se isto continuaría para valores máis grandes de n ou se é un caso puntual, como parecen apuntar as tendencias das gráficas. O máis probábel é que a causa principal desta situación non teña que ver coas optimizacións para os tipos de datos empregados e pola contra o que marque a diferenza sexa a existencia de instrucións que permiten executar as funcións `lapacke_ssyev` e `cblas_sgemm` máis eficientemente. Isto realmente non entra no alcance do noso traballo pero é un feito que sen dúbida ten un impacto claro nos resultados obtidos como se pode observar.

Análise da situación global

Non existe ningún contexto, no que o rendemento dos programas que traballan con `__bf16` sexa próximo ao da versión base, que traballa con `float`. Este tipo de dato xera un speedup minúsculo porque calquera mellora que podería introducir este tipo de dato vese opacada polo overhead que trae consigo. Este tipo de dato onde máis se destaca é nun contexto de precisión mixta como é o algoritmo `pca`, onde o groso do tempo de execución dedícase a operacións con outro tipo de dato e as operacións que se fan propiamente con `__bf16` non son só aritméticas.

O tipo de dato `_Float16` ten un rendemento mellor en ARM por ter soporte nativo e non existir un overhead para cada operación aritmética que se efectúa con este tipo de dato. O uso de instrucións que permiten traballar con estes datos posibilita, en case todos os contextos, ter un rendemento un 50 % superior ao mesmo programa pero que emprega `float`. Esta mellora tan substancial ocasiona que o cambio de tipo de dato sexa un factor a considerar para a mellora do rendemento dun programa, mellora que pode ser incrementada empregando programas optimizados para os tipos de datos, como se pode observar na Figura 4.7.

Tal e como se amosa na Figura 4.6 sen soporte é posíbel ter un rendemento mellor que o programa base, que traballa con `float`. Con todo, isto dáse nun contexto de precisión mixta (xunto a outros factores) pero fóra deste contexto resulta difícil atopar un programa que teña un rendemento próximo.

O tipo de dato `__fp16` é un dato que, a nivel de rendemento non destaca especialmente e sempre está por debaixo de `_Float16` ao tratarse dun tipo de dato adicado só ao almacenamento. Sen embargo, esta finalidade tamén podería supor unha vantaxe en contextos onde as operacións importantes non se poden realizar nunca con estes datos e teñen que empregarse buffers intermedios. Poderían ter cabida nun contexto de uso de operacións que requiren propiamente este tipo de dato para evitar o overhead. En xeral o uso deste tipo de dato é difícil de xustificar, a súa dispoñibilidade é moito máis limitada que `_Float16` e a súa

utilización non supón unha diferenza favorábel na grande maioría das situacións.

4.1.2. Métricas de precisión

Nesta subsección presentaremos os datos obtidos das métricas relativas á precisión dos datos e, seguindo unha organización similar ás métricas de desempeño: primeiro analizaremos as métricas individualmente e posteriormente faremos unha análise da situación xeral.

Como os tipos de datos `_Float16` e `__fp16` representan ambos *binary16* as diferenzas son case nulas, entran dentro da marxe de erro, polo que ambos tipos de datos serán tratados iguais e cando se dea unha diferenza o suficientemente significativa será resaltada.

Erro Medio Absoluto (MAE)

Algoritmo	Tipo dato	x86 (Intel)	ARM
AXPY	<code>_Float16</code>	0,00441966	0,00374735
	<code>__bf16</code>	0,03472626	0,03472626
	<code>__fp16</code>	—	0,00374735
DCT	<code>_Float16</code>	0,06348021	0,06348021
	<code>__bf16</code>	0,33723608	0,33723608
	<code>__fp16</code>	—	0,06348021
DWT_1D (LeGall)	<code>_Float16</code>	0,00167508	0,00160183
	<code>__bf16</code>	0,01329425	0,01329425
	<code>__fp16</code>	—	0,00160183
DWT_1D (CDF)	<code>_Float16</code>	0,00224679	0,00199024
	<code>__bf16</code>	0,02278251	0,02278251
	<code>__fp16</code>	—	0,00199024
PCA	<code>_Float16</code>	0,50399991	0,54743636
	<code>__bf16</code>	0,95178581	0,95895306
	<code>__fp16</code>	—	0,49282420
PCA_REIMPL	<code>_Float16</code>	1,13210577	1,13263199
	<code>__bf16</code>	1,18985001	1,19020325
	<code>__fp16</code>	—	1,13265033

Táboa 4.1: Erro absoluto medio, respecto aos resultados de *float*

En xeral pódese observar que, segundo os datos amosados na Táboa 4.1, o erro absoluto nos programas sinxelos, como AXPY e DWT_1D, ten valores que oscilan entre 10^{-3} e 10^{-2} , para o tipo de dato *binary16*, en ambas arquitecturas. Existe unha pequena diferenza no erro deste tipo de dato entre arquitecturas que sitúa á arquitectura x86, que non ten soporte nativo para as operación con estes tipos de datos, cun erro lixeiramente superior á arquitectura ARM, que conta con soporte. Destaca o feito de que no programa que implementa o algoritmo DWT_1D o cambio dun filtro *lossless* (sen perda) a un *lossy* (con perda) incrementa o erro cometido o que podería suxerir que uso destes tipos de datos aumenta as perdas que de por si xera o filtro.

Os programas algo máis complexos como o do algoritmo de DCT, xa teñen un erro de 6×10^{-2} que é bastante máis elevado pero podería ser aceptábel nalgún contexto. Este erro non é dramático pero hai que tomar en consideración o feito de que este programa realiza moitas operacións co tipo de dato *float*. Nun contexto de utilización de funcións adaptadas a traballar con *_Float16* ou *__fp16*, o erro podería escalar a valores que os descarten en escenarios sensíbeis pero, se están ben adaptadas, tamén podería reducir o erro a valores máis aceptábeis.

Para programas moi complexos como son os dous que implementan o algoritmo PCA a situación é inviábel directamente. Erros de aproximadamente 0,5, na versión tradicional, e 1,13, na versión que emprega a reimplementación de *lapacke_ssyeve*, son demasiado grandes para calquera contexto. Cabe destacar que este algoritmo é o único caso no que os resultados para *_Float16* e *__fp16* son diferentes, pero nas marxes mencionadas anteriormente. Isto seguramente estea motivado polo uso da función *cblas_hgemm* que traballa con *__fp16* que podería ter algunha implicación. Tamén podemos concluír que a reimplementación da función *lapacke_ssyeve* ten un impacto negativo no erro, moi probablemente porque os algoritmos empregados non están deseñados para traballar con esta precisión. Paradoxicamente esta reimplementación reduce a diferenza de erro cometido entre *_Float16* e *__fp16* en varias ordes de magnitude.

Para o tipo de dato *__bf16* o erro cometido en función do algoritmo ten o mesmo comportamento que nos tipos de datos anteriores, canto máis complexo é o programa máis grande adoita ser o erro cometido. Este tipo de dato presenta un erro maior que os de *binary16* pero a diferenza varía programa a programa, sendo mínima no algoritmo PCA_REIMPL. O MAE para *__bf16* coincide en ambas arquitecturas, salvo nos programas que implementan PCA, tal e como amosa a Táboa 4.1, esta diferenza non resulta relevante considerando o erro cometido.

Erro Cadrático Medio (MSE)

Para esta métrica pódese observar con bastante precisión que a potencia media do erro volve ser moi diferente entre os distintos algoritmos a analizar, como se observa na Táboa 4.2. Os algoritmos máis sinxelos incluso presentan diferenzas entre eles o que dificulta unha xeneralización por complexidade dos algoritmos. Por este motivo esta análise, aínda que xenérica, non ten por que ser aplicábel a outros programas de igual complexidade ou que realizan as mesmas operacións.

No programa que implementa o algoritmo AXPY atopamos diferenzas entre arquitecturas neste erro cadrático medio para *binary16*. En ambas arquitecturas sitúase na orde de 10^{-5} aínda que en ARM resulta aproximadamente un 30 % menor. Indicando que en ARM os picos de gran desviación son de menor magnitude, o que suxire que os outliers teñen menos impacto no cómputo global.

Para o que implementa DWT_1D para ambos filtros de wavelets, atopámonos cun MSE unha orde de magnitude inferior ao de AXPY. Ademais en ARM o MSE tamén mantense por debaixo do de x86, cunha redución dunha orde do

Algoritmo	Tipo dato	x86 (Intel)	ARM
AXPY	<i>_Float16</i>	0,00003565	0,00002497
	<i>__bf16</i>	0,00222205	0,00222205
	<i>__fp16</i>	—	0,00002497
DCT	<i>_Float16</i>	8,12039826	8,12039826
	<i>__bf16</i>	22,76057477	22,76057477
	<i>__fp16</i>	—	8,12039826
DWT_1D (LeGall)	<i>_Float16</i>	0,00000517	0,00000471
	<i>__bf16</i>	0,00032759	0,00032759
	<i>__fp16</i>	—	0,00000471
DWT_1D (CDF)	<i>_Float16</i>	0,00000898	0,00000697
	<i>__bf16</i>	0,00091504	0,00091504
	<i>__fp16</i>	—	0,00000697
PCA	<i>_Float16</i>	1,11783390	1,26825756
	<i>__bf16</i>	2,20790370	2,22397022
	<i>__fp16</i>	—	2,22397022
PCA_REIMPL	<i>_Float16</i>	2,00913799	2,01110287
	<i>__bf16</i>	2,22103838	2,22270939
	<i>__fp16</i>	—	2,01115495

Táboa 4.2: Erro cadrático medio, respecto aos resultados de *float*

10 % aproximadamente para o filtro LeGall e do 25 % para o CDF. Por último obsérvase que cando se emprega un filtro “lossless” (sen perda), como é o de LeGall, produce un MSE menor que un “lossy” (con perda) como é o de CDF. Isto suxire que a baixa precisión dos tipos de datos suxeitos a esta análise amplifican as perdas que xa introduce o propio procesado da wavelet.

No algoritmo DCT a obsérvase un comportamento distinto ao de casos anteriores: o MSE é moi elevado e é igual nas dúas arquitecturas. En concreto o erro cadrático medio dispárase até $\sim 8,1$ indicando a existencia picos de gran desviación nos resultados unha forte influencia dos outliers no cómputo global. Este resultado é esperábel xa que se atendemos ao códigos e emprega precisión mixta nas operacións trigonométricas, núcleo do algoritmo, provocando continuos cambios de formato e redondeos frecuentes.

Os programas que implementan o algoritmo de PCA, con ou sen a reimplementación de `lapacke_ssyev`, teñen un MSE menor que o algoritmo DCT pero moitas ordes de magnitude superior a AXPY e DWT_1D. Volve haber diferenzas no MSE para *binary16* entre as arquitecturas x86 e ARM. Neste caso a diferenza é un 13 % superior para ARM na versión sen a reimplementación e case nula para a versión coa reimplementación. Respecto ao valor do MSE destaca a súa duplicación ao empregar a función reimplementada, suxerindo que esta maior cantidade de operacións con estes tipos de datos introduce máis erros. Neste aspecto resulta moi interesante destacar que o tipo de dato *__fp16* ten un MSE un 75 % superior ao de *_Float16* na versión sen reimplmentación pero na versión con reimplmentación son practicamente iguais. Chama tamén a atención que o MSE aumente máis en x86 que en ARM. Isto podería suxerir que a non utilización de

instrucións aritméticas propias para este tipo de dato causa un maior MSE por operación realizada. Esta podería ser tamén a xustificación a por que non é igual o MSE nas dúas arquitecturas pese a representar a mesma información.

Para o tipo de dato `__bf16` o erro cometido en función do algoritmo ten o mesmo comportamento que nos tipos de datos anteriores, canto máis complexo é o programa máis grande adoita ser o erro cometido agás DCT. Presenta para todos os algoritmos un MSE dúas ordes de magnitude superior a *binary16*, cando o erro é pequeno. Para DCT é 2,75 veces maior, na versión de PCA sen reimplementación é case o dobre e na versión de PCA con reimplementación é só un 10 % superior. Isto podería suxerir que a menor mantisa deste tipo de dato amplifica os outliers dos redondeos frecuentes, como en DCT, e dos buffers intermedios, como en PCA e PCA_REIMPL. Con todo, ao empregar funcións adaptadas a este tipo de dato e prescindir dos buffers intermedios os picos de gran desviación resultado da influencia dos outliers, aumentan menos proporcionalmente que para *binary16*. Tamén é importante destacar que que o MSE non é igual en x86 e en ARM en ambas implementacións de PCA e, do mesmo xeito que pasaba con `_Float16`, o MSE é menor en x86 sendo case igual en PCA_REIMPL.

O MAE e o MSE polo xeral coinciden en sinalar os mesmos comportamentos (maior erro en PCA e PCA_REIMPL, menor en AXPY e DWT_1D), pero cada un aporta unha perspectiva distinta. O MAE amosa a desviación media sen dar peso extra aos outliers, polo que pasa por alto os picos de gran erro de DCT que, en contraste, o MSE detecta con claridade. Esta combinación de visión lineal e sensibilización aos valores extremos ofrece unha análise máis completa do impacto real do uso dos tipos de datos probados.

Raíz do Erro Cadrático Medio Normalizada (NRMSE)

A raíz do erro cadrático medio normalizado expresa a raíz do MSE dividida polo rango (diferenza entre valor máximo e mínimo) da saída, xerando unha magnitude adimensional que expresa o erro relativo. Esta métrica varía notabelmente entre algoritmos, como se pode ver na Táboa 4.3, e homoxeneiza todos os erros nunha mesma escala, permitindo a comparación directa entre algoritmos.

Nos programas que implementan algoritmos sinxelos, como son AXPY e DWT_1D, o NRMSE cometido é moi baixo, da orde de 0,01 % para *binary16*. Na arquitectura x86 obsérvase un lixeiro incremento do erro pero sen variar a orde de magnitude, polo que a diferenza é desprezábel. Entre ambos algoritmos, AXPY presenta un NRMSE algo maior que DWT_1D; esta diferenza agrándase co uso do filtro *lossy*. As discrepancias entre AXPY e DWT_1D resultan máis marcadas en x86 que en ARM, sen superar en ningún caso a mesma orde de magnitude.

No caso de DCT, un algoritmo de complexidade media, atopámonos cunha situación bastante distinta á anterior. Observamos un incremento notábel do erro relativo: o NRMSE para *binary16* acada o 0,45 %, unha orde de magnitude superior. Este valor non varía entre arquitecturas, isto pode vir motivado polo

Algoritmo	Tipo dato	x86 (Intel)	ARM
AXPY	<i>_Float16</i>	0,00017790	0,00014888
	<i>__bf16</i>	0,00140447	0,00140447
	<i>__fp16</i>	—	0,00014888
DCT	<i>_Float16</i>	0,00457225	0,00457225
	<i>__bf16</i>	0,00765478	0,00765478
	<i>__fp16</i>	—	0,00457225
DWT_1D (LeGall)	<i>_Float16</i>	0,00010938	0,00010434
	<i>__bf16</i>	0,00087037	0,00087037
	<i>__fp16</i>	—	0,00010434
DWT_1D (CDF)	<i>_Float16</i>	0,00013396	0,00011804
	<i>__bf16</i>	0,00135217	0,00135217
	<i>__fp16</i>	—	0,00011804
PCA	<i>_Float16</i>	0,06923250	0,07374280
	<i>__bf16</i>	0,09729969	0,09765183
	<i>__fp16</i>	—	0,06882992
PCA_REIMPL	<i>_Float16</i>	0,09281674	0,09286093
	<i>__bf16</i>	0,09758868	0,09762414
	<i>__fp16</i>	—	0,09286214

Táboa 4.3: Raíz do erro cadrático medio normalizada, respecto aos resultados de *float*

uso de precisión mixta para realizar as operacións trigonométricas. O que apunta a que o uso de precisión mixta causa un maior erro relativo, ao mesmo tempo que homoxeneiza o erro entre arquitecturas.

Para o algoritmo PCA, tanto na súa implementación orixinal como na variante coa reimplementación de `lapacke_ssyev`, o NRMSE acada valores excesivamente altos: arredor do 7 % en PCA e case do 10 % en PCA_REIMPL. Este incremento débese á maior complexidade algorítmica respecto de DCT e AXPY, ás operacións de descomposición espectral e á xestión de buffers intermedios. Neste o NRMSE é diferente entre arquitecturas, onde x86 presenta erros lixeiramente menores. Esta diferenza é particularmente reducida para PCA_REIMPL, onde é desprezável. Ademais, obsérvase que o NRMSE para o tipo de dato *__fp16* difire do de *_Float16*. Na implementación orixinal de PCA é menor que para *_Float16* e para a variante coa reimplementación é aproximadamente igual.

Con *__bf16* obsérvanse tendencias similares ás de *binary16* coa complexidade dos algoritmos, pero cun NRMSE maior. En AXPY a diferenza é dunha orde de magnitude, e a medida que aumenta a complexidade dos algoritmos a diferenza redúcese, aínda que nunca se equipara á de *binary16*. Ademais, estes valores mantéñense iguais en x86 e ARM, agás en PCA e PCA_REIMPL que é desprezável pero existe.

Con todos estes datos pódese afirmar que, conforme incrementa complexidade dos algoritmos o erro proporcional cometido é maior ao empregar estes tipos de datos. O erro cometido ao empregar *binary16* é de 0,01 % en algoritmos sinxelos o que posibilita o seu uso neste contexto pero en DCT e, especialmente, en PCA

é demasiado elevado como para permitir o seu uso en calquera contexto. Pola súa parte o erro mínimo cometido ao empregar `__bf16` é de aproximadamente 0,1 % o que fai que nalgúns contextos onde a precisión non sexa fundamental poida ser empregado, pero como o erro máximo observado chega a ser do 10 % a decisión debería ser dependente do contexto.

En conxunto, podemos concluír que o NRMSE crece coa complexidade do algoritmo. Nos algoritmos sinxelos o erro con `binary16` mantense en torno ao 0,01 %, resultando asumíbel en tarefas que toleren pequenas perdas de precisión. No caso do DCT, o NRMSE elévase a aproximadamente 0,45 %, un valor que xa non se podería considerar baixo, e con PCA acada valores entre o 7 % e o 10 %, limitando drasticamente a súa utilidade en contextos de alta precisión. Con `__bf16` obsérvase un incremento máis suave do erro coa complexidade dos algoritmos, tendo un 0,1 % nos sinxelos como AXPY e ata 9,8 % nos complexos como PCA. Isto fai que este tipo de dato se poda empregar nalgúns contextos pero tería que estudarse o caso concreto. En consecuencia, a elección do uso destes tipos de datos debe facerse en función do nivel de precisión esixido: `binary16` resulta viábel en escenarios de alta precisión, mentres que `__bf16` debería estudarse para o caso concreto. Ningún dos tres tipos de datos é recomendábel en operacións tan sensíbeis como son as da PCA, tomando en conta que o erro mínimo é de 6,9 %.

Coeficiente de Correlación de Pearson

Algoritmo	Tipo dato	x86 (Intel)	ARM
AXPY	<code>_Float16</code>	0,99999969	0,99999979
	<code>__bf16</code>	0,99998002	0,99998002
	<code>__fp16</code>	—	0,99999979
DCT	<code>_Float16</code>	0,91499923	0,91499923
	<code>__bf16</code>	0,56606664	0,56606664
	<code>__fp16</code>	—	0,91499923
DWT_1D (LeGall)	<code>_Float16</code>	0,99999983	0,99999985
	<code>__bf16</code>	0,99998949	0,99998949
	<code>__fp16</code>	—	0,99999985
DWT_1D (CDF)	<code>_Float16</code>	0,99999973	0,99999979
	<code>__bf16</code>	0,99997881	0,99997881
	<code>__fp16</code>	—	0,99999979
PCA	<code>_Float16</code>	0,44314329	0,36830004
	<code>__bf16</code>	0,00508984	−0,00214169
	<code>__fp16</code>	—	0,44967213
PCA_REIMPL	<code>_Float16</code>	−0,00087247	−0,00168957
	<code>__bf16</code>	−0,00090512	−0,00171632
	<code>__fp16</code>	—	−0,00168933

Táboa 4.4: Coeficiente de correlación de Pearson, respecto aos resultados de `float`

A continuación amosamos o coeficiente de correlación de Pearson (r) para avaliar ata que punto os resultados xerados concordan cos obtidos empregando `float`.

O coeficiente de Pearson, que oscila entre -1 e 1 , mide a correlación lineal entre dúas mostras: valores próximos a 1 indicarán que o tipo de dato conserva practicamente intacta a forma do sinal. Para unha comparación directa, empregaremos $|r|$ xa que o signo só reflicte un cambio de dirección na relación. Por exemplo se temos $|r| = 0,95$ entón $r^2 \approx 0,90$: só coincide o 90 % da varianza lineal (varianza explicada), hai perdas por redondeo dun 10 %.

Tal como se pode apreciar na Táboa 4.4 o coeficiente de correlación varía substancialmente segundo o algoritmo e o tipo de dato. Tendo en conta que a perda de información escala con r^2 , incluso pequenas diferenzas poden traducirse en oscilacións do erro de gran magnitude.

Para o algoritmo AXPY, o coeficiente de correlación de *binary16* é bastante próximo a 1 para ambas arquitecturas. Isto indica que o grao de conservación da estrutura lineal do sinal é case completo, cunha perda desprezábel. Hai unha ínfima diferenza entre as arquitecturas x86 e ARM pero non supón unha diferenza destacábel alén do feito de que non é unha coincidencia completa.

O algoritmo DWT_1D experimenta unha situación similar a AXPY, os coeficientes de correlación para *binary16*, en ARM, son moi próximos a 1 . Como sucedeu en AXPY hai unha diferenza ínfima (10^{-7}) entre x86 e ARM. En liña con observado noutras métricas neste algoritmo hai unha conservación da estrutura lineal do sinal superior a AXPY, aínda que esta diferenza é marxinal (10^{-7}).

No algoritmo de DCT afastámonos dos coeficientes de correlación próximos a 1 : con *binary16*, situase en $0,915$ aproximadamente en ambas arquitecturas. Isto equivale a unha varianza explicada de só o 84 %, pérdese aproximadamente un 16 % da información lineal respecto a *float*, unha caída moi significativa en comparación cos algoritmos sinxelos. Con todo, esta correlación ao ser maior de $0,9$ pode considerarse moi forte.

Tanto o PCA orixinal como a súa reimplementación amosan coeficientes de correlación moi baixos e nalgúns casos negativos, o que indica unha conservación case nula da estrutura lineal. En PCA orixinal *_Float16* acada $r \approx 0,44$ en x86 (correlación moderada) e $r \approx 0,37$ en ARM (correlación feble), mentres *__fp16* queda con $r \approx 0,45$. Na reimplementación, todos os formatos teñen un $|r| < 0,002$, confirmando que non se mantén relación lineal e de feito o mantemento é moito peor que na versión orixinal. En termos de varianza explicada, isto corresponde a $r^2 < 0,2$, é dicir, unha perda superior ao 80 % da información lineal orixinal o que a efectos prácticos supón a perda total de correlación. Ademais, as variacións entre arquitecturas e tipos de datos evidencian a alta sensibilidade de PCA ao redondeo a 16 bits. Destes resultados pódese concluír que *lapacke_ssyev* non foi deseñada para formatos de 16 bits; coa súa reimplementación para empregar estes tipos de datos, os seus algoritmos internos non manteñen a precisión lineal necesaria.

Con *__bf16* obsérvanse tendencias similares ás de *binary16* pero agravadas co crecemento da complexidade dos algoritmos. Para AXPY e DWT_1D manteñen un coeficiente de correlación moi preto de 1 ($r \approx 0,99998$), pero a distancia de

$\sim 10^{-5}$ fronte aos $\sim 10^{-7}$ de *binary16* implica unha perda de información catro ordes de magnitude maior. Para o algoritmo de DCT o coeficiente de correlación descende a $r \approx 0,57$, fronte a $r \approx 0,91$, que supón unha redución drástica. En PCA atopámonos cun coeficiente de correlación dúas ordes de magnitude máis baixo que para *binary16* que indica unha correlación nula. No caso de PCA_REIMPL a diferenza redúcese e os coeficientes de correlación rematan sendo aproximadamente iguais e igualmente indicando unha correlación inexistente. En conxunto, pódese afirmar que `__bf16` amplifica as perdas de correlación lineal e demostra unha maior sensibilidade ao redondeo a 16 bits que *binary16* para todos os casos.

O coeficiente de correlación de Pearson ao non medir magnitudes de erro senón fidelidade á forma lineal orixinal, cobre un aspecto que o MAE, MSE e NRMSE non alcanzan. Por exemplo para *binary16*, en AXPY e DWT_1D o MAE ($\sim 10^{-7}$) e o MSE ($\sim 10^{-14}$) confirman desviacións mínimas, e o NRMSE ($\sim 10^{-4}$) mostra que ese erro é insignificante respecto ao rango do sinal; con todo, $r \approx 0,9999997$ subliña adicionalmente que a forma do sinal consérvase case intacta. En PCA e PCA_REIMPL, tamén para *binary16*, onde MAE/MSE/NRMSE amosan erros en torno ao 20 ou 25 % do rango. É con Pearson cando se evidencia a ruptura case total da correlación lineal no algoritmo PCA, algo que o MAE, mesmo moderado, non deixa ver. Integrar Pearson cunha avaliación de MAE, MSE e NRMSE ofrece así unha radiografía completa da precisión das operacións con estes datos.

Análise da situación global

Para o tipo de dato `__bf16` obsérvase que o redondeo a este formato agrava todas as perdas: os erros en AXPY e DWT_1D aumentan varias ordes de magnitude sobre *binary16*, e a correlación sempre é menor, destacando en PCA onde pasa de correlación moderada a nula. Con todo, o deterioro coa complexidade do algoritmo (e co uso de funcións nativas) é moito menor que para *binary16* sendo no caso máis baixo practicamente igual en cuanto a erros e mantemento de correlación. Todo isto ten sentido posto que este tipo de dato ten unha mantisa moito máis pequena que *binary16* e ten un maior expoñente. Isto amosa que, a pesar de ser competitivo en tarefas relativamente sinxelas, non é axeitado para algoritmos que demanden mantemento fiel da estrutura lineal.

Os tipos de datos *binary16* ofrecen un bo compromiso de precisión en operacións sinxelas tendo erros mínimos e unha fidelidade do sinal case total. Con todo, en algoritmos máis complexos como DCT perde arredor dun 16 % da varianza lineal, que sumado ao erro que comete vólveo pouco aconsellábel, e en PCA a correlación case desaparece. Todo isto revela as limitacións destes tipos de datos en procesos complexos que requiren conservación estrita da relación lineal.

Capítulo 5

Conclusións e posíbeis ampliacións

Neste capítulo final resúmense os principal plantexamento do traballo e as conclusións extraídas da experimentación realizada no capítulo anterior. Ademais, comentaranse varias posibilidades de continuación que admite este traballo.

Neste traballo analizouse o uso dos formatos de punto flotante de precisión reducida *binary16* e *bfloat16* en arquitecturas de CPUs. Este plantexamento responde á escaseza de ferramentas de avaliación comparativa específicas para este tipo de hardware, fronte á súa ampla dispoñibilidade en GPUs. Para abordar esta carencia, desenvolveuse un benchmark en linguaxe C co obxectivo de avaliar o desempeño destes tipos de datos nas plataformas x86 e ARM. As probas incluídas no benchmark están deseñadas para explorar escenarios con diferentes niveis de complexidade algorítmica e de demanda de memoria. Estas probas proporcionan unha visión sistemática das vantaxes e limitacións dos formatos de precisión reducida en entornos de computación baseados en CPUs.

As métricas empregadas permiten ver o rendemento dos tipos de datos estudados e contextualizalos coa precisión que ofrecen dende varias perspectivas permitindo detectar non só a falta de precisión senón contextualizala. Esta contextualización permite ver como é o erro que se comete ao empregar os tipos de datos, detectar a presenza ou non de outliers e ver a perda das tendencias e patróns dos datos.

A partir dos resultados obtidos no benchmark plantexado anteriormente pódense extraer tres conclusións principais:

1. O uso de *bfloat16* non é recomendábel en ningún contexto no que non exista soporte nativo para as operacións debido ao extremadamente baixo rendemento e agravado pola falta de precisión en algoritmos complexos.
2. *binary16* resulta un tipo de dato axeitado en precisión e rendemento para algoritmos de baixa complexidade se existe soporte nativo para as operacións.

3. A utilización de funcións específicas optimizadas para os tipos de datos xera un mellor rendemento pero, se non están adaptadas correctamente, pode aumentar a perda de información. Se as funcións só son deseñadas para mudar o tipo de dato co que operan, sen adaptar a lóxica interna á menor precisión dos tipos de datos, o seu uso resultará nunha perda de información substancial.

Como traballo futuro neste campo, consideramos necesario destacar a necesidade de volver realizar este experimento polo menos empregando procesadores con soporte nativo para operacións en *bfloat16*, o que permitiría unha análise máis rigorosa deste formato. O experimento beneficiaríase tamén de contar con plataformas hardware que ofrezan soporte completo para todos os tipos de datos de precisión reducida nas distintas arquitecturas de estudo, por exemplo un procesador Intel Sapphire Rapids e un ARMv8.6-A. Isto posibilitaría a realización dunha comparativa directa entre execucións con e sen soporte nativo dentro da mesma arquitectura, facilitando a identificación de diferenzas significativas no rendemento e precisión.

En liña co anterior, tamén sería valioso estender este experimento a outras arquitecturas de CPUs que soportan ambos tipos de datos, como RISC-V, para realizar comparacións entre arquitecturas e dentro da arquitectura co soporte activado ou non. Este enfoque permitiría avaliar o impacto das diferenzas de deseño das arquitecturas no rendemento e precisión, conectando coas metodoloxías de benchmarking orientadas a identificar fortalezas e debilidades entre plataformas.

Relativo aos programas empregados unha boa ampliación cara un futuro experimento pasaría por implementar o algoritmo de PCA de xeito completamente nativo para `_Float16`, `__fp16` e `__bf16`. Isto implicaría reimplementar a función `cblas_sgemv` para estes tipos de datos e adaptar os algoritmos desta (e de `lapacke_ssyev`) para que poidan traballar con estes datos minimizando a perda de información. Este novo programa permitiría realizar un estudo completo do impacto dos tipos de datos nunha situación dun algoritmo complexo completamente adaptado aos tipos de datos probados, empregando só a memoria necesaria e facendo todas as operacións con eles.

Apéndice A

Manuais técnicos

Todos os ficheiros empregados neste traballo poden consultarse no seguinte repositorio de GitHub:

<https://github.com/Espasant3/Benchmark-reduced-precision-float>.

A.1. Estrutura de ficheiros

O repositorio con todos os programas do traballo está estruturado dun xeito modular, buscando facilitar as modificacións ou posibles ampliacións. A súa organización é a seguinte:

- **Bibliografía:** Directorio onde se almacena información relevante para o traballo, ben en forma de páxinas con información, problemas atopados durante o desenvolvemento ou arquitecturas soportadas.
- **Metrics:** Directorio que almacena todo o necesario para a elaboración das métricas.
 - **Datos:** directorio que almacena todos os datos cos que se poden elaborar as métricas.
 - **Resultados-ejecucion:** directorio que almacena, en directorios separados por arquitectura, todos os datos dos resultados das execucións dos programas elaborados co formato:
`<programa>_n<valor_n>_seed<valor_seed>.csv`.
Almacena tamén nun directorio interno con nome “**Datos-previos-operaciones**” os datos antes das operación co mesmo formato de nome pero co prefixo “**source_**” ¹.

¹En Intel tamén existe un directorio chamado “Discarded”, con datos descartados e non empregados, pero válidos

- **Tiempos-ejecucion:** directorio que almacena, en directorios separados por arquitectura, todos os tempos de execución dos programas elaborados co formato `tiempos_<programa>.def.csv`. Contén tamén un directorio “**All-data**” que contén os datos dos que se extraen estos tempos, posto que os datos definitivos son a mediana de tempos se houbera varios tempos para un programa.
- **Metricas-calculadas:** directorio que contén as métricas elaboradas cos datos anteriores, separadas por arquitectura. Contén os directorios: **MAE**, **MSE**, **NRMSE**, **Pearson**, **Speedup** e **Unused**. No directorio **Unused** almacénanse métricas que orixinalmente se calcularon pero remataron por non empregarse.
- **Programas:** directorio que contén todos os programas relacionados coas métricas, a súa elaboración e a obtención dos datos cos que se crean ditas métricas.
 - *MAE.py*, *MSE.py*, *NRMSE.py*, *Pearson.py* e *speedup.py*: programas que calculan as métricas que lles dan nome a partir de ficheiros csv co formato dos almacenados.
 - *Relacion-de-compresion.py* e *SSIM.py*: igual que os anteriores pero estas métricas foron descartadas, por tanto, os ficheiros poden precisar algunha modificación.
 - *resultado_ejecucion.py*: programa para extraer os resultados da execución dun programa indicando gardando os datos en formato “Ejecucion,Indice,Valor” para vectores, mentres que para matrices é “Ejecucion,Fila,Columna,Valor”. O campo “Ejecucion” servirá para a obtención de varios datos nun só programa como no caso de DWT_1D. Adicionalmente pode extraer dos datos antes das operacións cun flag.
 - *tiempo_ejecucion.py*: programa para extraer o tempo de execución dun programa gardando os datos en formato “*n*, *Tiempo_1*, *Tiempo_2*, ... , *Tiempo_k*”, habendo tantos *Tiempo_k* como execucións do programa, detectadas en función das veces que se imprime por pantalla “Tiempo_ejecucion”. Os tempos son o resultados da mediana dun número especificados de execucións, 11 por defecto, para unha lista crecente de valores de *n* e cun seed determinado, 1234 por defecto. Os ficheiros xerados terán un nome que indicará o valor de *n* inicial e final, o número de puntos (tamaño da lista) e a data actual en formato “%Y%m%d_%H%M%S”. Isto permite realizar as execucións sen medo a que se sobrescriban os datos.
 - *get_all_times.sh* e *get_all_results.sh*: ficheiros que automatizan a obtención dos tempos de execución e resultados chamando aos

programas *tiempo_ejecucion.py* e *resultado_ejecucion.py*, respectivamente, para todos os programas do directorio **Programas**.

- *mediana_csv_tiempos.py*: programa que elabora a mediana de varios ficheiros csv co formato de almacenamento indicado anteriormente.

■ **Programas**: Directorio que contén todos os programas que implementan os algoritmos estudados neste traballo.

• **AXPY, DCT, DWT_1D, PCA e PCA_REIMPL**: directorios que conteñen os programas que implementan os algoritmos que lles dan nome. Conteñen 3 tipos de ficheiros:

- *<nome>_<dato>.c*: programa que implementa o algoritmo do directorio no que se atopa para o tipo de dato que indica. “<nome>” terá o mesmo nome que o directorio no que se atopa pero en minúscula. “<dato>” poderá ser FP32, FP16, FP16_ARM ou BF16 segundo o tipo de dato empregado sexa *float*, *_Float16*, *__fp16* ou *__bf16*, respectivamente. Dentro destes programas enténdese como **programa principal**, inicial, base ou orixinal o que emprega *float*, é dicir, o **programa** *<nome>_FP32* e o resto serán as súas versións. Isto ven motivado polo feito de que é o primeiro programa implementado para cada algoritmo e que despois é adaptado para empregar outros tipos de datos.
- *<nome>_compile_<target>.sh*: Script para compilar os programas dese directorio, para a arquitectura de destino “<target>”. Ten o mesmo nome que os programas anteriores e as arquitecturas “<target>” recoñecidas son “Intel”, “AMD” e “ARM” (non diferencia entre provedores de CPU). Todos os programas son compilados sempre cos flags *-Wall -g*, que son empregados principalmente para asegurar o correcto funcionamento dos programas, e non teñen un impacto no rendemento.
- *<nome>_run_<target>.sh*: Script que executa todos os programas dese directorio, para a arquitectura de destino “<target>”. Ten o mesmo nome que os programas anteriores e as arquitecturas “<target>” recoñecidas son “Intel”, “AMD” e “ARM”.

Adicionalmente dentro do directorio PCA_REIMPL atopámonos co directorio **functions-adapted** que contén todos os ficheiros precisos para a execución da nova función *lapacke_hfsyev*. Dentro deste directorio hai 4 directorios: **src**, **fortran_sourced**, **include** e **utils**. Destacando que os ficheiros *.h*, necesarios para a utilización das funcións, atópanse no directorio **include**. O contido deste directorio pode consultarse na Sección A.2.

- *compile_all.sh*: ficehiro global que compila todas os programas empregando o script de execución concreto para a nosa arquitectura e procesador, detectada mediante o comando `uname -m` e a información do ficheiro “`/proc/cpuinfo`”.
- **Tests-Python**: directorio que contén os programas python empregados para verificar a correcta implementación dos algoritmos en linguaxe C. Contén tamén un ficheiro “`requirements.txt`”, cos paquetes necesarios, e un script “`setup-python-env.sh`” para crear un entorno de python que permita executar todos os programas deste directorio e do traballo en xeral.
- **Unused-Examples**: directorio que contén programas que elaboraron pero nalgún punto foron descartados por falta de tempo ou desviarse do enfoque. Aquí por exemplo van algunhas métricas contempladas inicialmente pero descartadas.
- *Dockerfile*: ficheiro para a creación dunha imaxe de contedor Docker que permita crear rápida un entorno no que traballar.

A.2. Montaxe e arranque do proxecto

Como método xeral, a forma máis fácil de poder traballar e probar este proxecto é a seguinte:

1. Clonar o proxecto:

```
git clone git@github.com:Espasant3/Benchmark-reduced-precision-float.git
```

2. Cambiar ao directorio raíz:

```
cd Benchmark-reduced-precision-float
```

3. Construír a imaxe de Docker:

```
docker build -t tfg-benchmark-dev-env .
```

4. Arrancar o contedor:

```
docker run -it --rm -v $(pwd):/workspace tfg-benchmark-dev-env
```

Directorios functions-adapted

Directorio que contén todos os programas que precisa `lapacke_hfsyev` para o seu funcionamento. Este directorio pertence ao programa `PCA_REIMPL`, que contén exactamente os seguintes ficheiros:

```
functions-adapted
+-- fortran_sourced
|   +-- hfcopy.c
|   +-- hfdot.c
|   +-- hfgemm.c
|   +-- hfgemv.c
|   +-- hfger.c
|   +-- hflacpy.c
|   +-- hflae2.c
|   +-- hflaev2.c
|   +-- hflamch_half_precision.c
|   +-- hflanst.c
|   +-- hflansy.c
|   +-- hflapy2.c
|   +-- hflarf1f.c
|   +-- hflarf1l.c
|   +-- hflarfb.c
|   +-- hflarfg.c
|   +-- hflarft.c
|   +-- hflartg.c
|   +-- hflascl.c
|   +-- hflasct.c
|   +-- hflasr.c
|   +-- hflasrt.c
|   +-- hlassq.c
|   +-- hflatrd.c
|   +-- hfnrm2.c
|   +-- hforg2l.c
|   +-- hforg2r.c
|   +-- hforgql.c
|   +-- hforgqr.c
|   +-- hforgtr.c
|   +-- hfscal.c
|   +-- hfsteqr.c
|   +-- hfsterf.c
|   +-- hfswap.c
|   +-- hfsyev.c
```

```
|  +-- hfsymv.c
|  +-- hfsyr2.c
|  +-- hfsyr2k.c
|  +-- hfsytd2.c
|  +-- hfsytrd.c
|  +-- hftrmm.c
|  +-- ilaenv_reimpl_half_precision.c
|  +-- ilaslc_reimpl_half_precision.c
|  \-- ilaslr_reimpl_half_precision.c
+-- include
|  +-- definitions.h
|  +-- lapacke_utils_reimpl.h
|  \-- pca_reimpl.h
+-- src
|  +-- lapacke_hfsyev.c
|  \-- lapacke_hfsyev_work.c
\-- utils
    +-- c_xerbla.c
    +-- hfroundup_lwork.c
    +-- ieeeck_reimpl_half_precision.c
    +-- iparmq_reimpl.c
    +-- lapacke_hfge_trans.c
    +-- lapacke_hfsy_nancheck.c
    +-- lapacke_hfsy_trans.c
    +-- lapacke_hftr_nancheck.c
    +-- lapacke_hftr_trans.c
    +-- lapacke_nancheck_reimpl.c
    +-- lapacke_xerbla.c
    \-- lsame_reimpl.c
```

Apéndice B

Manuais de usuario

Este manual redactouse para un sistema Linux coa distribución Ubuntu 24.04, sen embargo, debería ser válido para a maioría de sistemas Linux.

B.1. Compilación dos programas

Para a compilación completa dos programas, polo menos a primeira vez, recoméndase empregar directamente o script *compile_all.sh* que xa detecta o target de compilación e dá os avisos pertinentes en caso de situación non soportada.

En xeral é recomendábel executar os scripts de compilación de cada directorio dos programas para realizar a compilación e non ter ningún problema.

Todos os scripts poden recibir como argumentos os seguintes flags (o script xeral só pasará os flags aos scripts que invoca):

- **-f** ou **-force**: Ao rematar as compilacións normais se realiza a compilación cruzada dos programas para ARM de 64 bits. Non se fai para os programas que implementan o algoritmo de pca porque **cblas_hgemm** é unha función exclusiva de ARM.
- **-h** ou **-help**: Imprime a axuda do script.
- **flags de compilación adicionais**: flags de GCC adicionais que se quere empregar para a compilación dos programas. O script non verifica en ningún momento estes flags, se non son validados a compilación con GCC dará erros.

B.2. Execución dos programas

Pese á existencia do script xeral de execución *run_all.sh*, non se recomenda empregalo se non se desexa comprobar algún cambio concreto en todos os programas á vez.

Para executar un programa á vez será necesario facelo a man, e para executar todos os programas dun algoritmo en particular poderase empregar o script de execución do directorio no que están os programas (previa execución do script de compilación).

Estes scripts de execución consisten nun bucle que percorre todos os ficheiros executábeis e se cumpren un criterio execútaos cos argumentos recibidos.

Todos os scripts poden recibir como argumentos os seguintes flags (no script global os flags serán transmitidos a cada script de execución específico):

- **tamanho_N**: Argumento obrigatorio. É o primeiro argumento recibido polo script. Compróbase que se teña recibido e que sexa positivo e maior de 0.
- **seed**: Argumento opcional. É o segundo argumento recibido polo script. En caso de ser recibido compróbase que sexa positivo e maior de 0.
- **-f** ou **-force**: Ao rematar as execucións normais se realiza a execución emulada dos programas que anteriormente se teñan compilado de xeito cruzado.
- **-m** ou **-memcheck**: Activa a execución con Valgrind dos programas empregando o comando `valgrind` coas seguintes opcións `-tool=memcheck -leak-check=full -show-leak-kinds=all -track-origins=yes -s`. Isto só se executa para os programa na súa execución normal, non na emulada.
- **-v** ou **-verbose**: Os ficheiros recibirán adicionalmente o argumento `-v` cando se executen, indicando que se trata da opción verbose.
- **-h** ou **-help**: Imprime a axuda do script.

B.3. Argumentos dos programas

Todos os programas que implementan os algoritmos reciben un argumento obrigatorio, un argumento opcional e un flag opcional do seguinte xeito: `axpy_<tipo_de_dato> <tamanho N> [<seed>] [-v]`.

tamanho N : Obrigatorio. Lonxitude dos arrays recollida internamente mediante a función `atoi` de `stdlib.h`.

seed : Opcional. Semente para `rand()` recollida internamente mediante a función `atoi`.

-v : Opcional. Modo detallado (verbose) que amosa os datos antes e despois das operacións. Xestionado internamente coa función `getopt` de `unistd.h`.

B.4. Obtención de datos

Para a obtención tanto dos tempos de execución coma dos datos de execución empregaranse dous scripts python, polo que se recomenda empregar o entorno que se crea con script `setup-python-env.sh` do directorio **Test-Python**.

B.4.1. Obtención dos tempos de execución

A obtención dos tempos de execución realízase co script `tiempo_ejecucion.py` que extrae os tempos de execución dun programa dado e os almacena nun formato tratado. O formato é “Ejecucion,Indice,Valor” para vectores, mentres que para matrices é “Ejecucion,Fila,Columna,Valor”. Acepta varios tempos de execución por programa e detecta os tempos de execución capturando a saída e buscando a liña “Tiempo de ejecucion:” e recollendo o tempo que se imprime a continuación. A cantidade de tempos de execución detéctase cunha execución inicial para un valor de n pequeno. O programa execútase así e conta coas seguintes opcións:

```
python tiempo_ejecucion.py <programa> [opcions]
```

programa Ruta ao executábel do programa (obrigatorio).

-seed <int> Semente para o programa a executar. Valor defecto: 1234.

-num_ejecuciones <int> Número de veces que se mide o tempo do programa para cada tamaño de n antes de facer a mediana. Valor por defecto: 11.

-save_data <true|false> Guarda os resultados nun ficheiro CSV. Valor por defecto: `false`.

-save_plot <true|false> Exporta a gráfica resultante a formato PNG. Valor por defecto: `false`.

-show_plot <true|false> Amosa a gráfica por pantalla. Valor por defecto: `false`.

-n_extended Flag que, de estar presente, fai que se empregue a lista de tamaños de n extendida.

Ao rematar, os datos almacénase nun ficheiro CSV co seguinte formato de cabeceira:

```
n,Tiempo_1,Tiempo_2,Tiempo_3,...,Tiempo_k
```

onde n é o tamaño de n co que se executou o programa e cada `Tiempo_i` corresponde a unha das k execucións realizadas. Así, cada fila do CSV recolle o conxunto completo de tempos medidos para un valor de n .

Unha vez xerado, o ficheiro móvese á carpeta correspondente baixo a ruta

/Metricas/Datos/Tiempos-ejecucion/<vendor>/All-data/

onde <vendor> escóllese en base á arquitectura e ao ficheiro `/proc/cpuinfo`:

- Arquitectura `x86_64` (`amd64` ou `AMD64`):
 - **Intel**: Aparece “intel” ou “genuineintel”.
 - **AMD**: Aparece “amd” ou “authenticamd”.
 - **x86_64**: Non aparece ningún dos valores anteriores.
- Arquitectura `AArch64`: Ten sempre valor **ARM** independentemente do contido do ficheiro `/proc/cpuinfo`.

Se non se a arquitectura non está soportada, o CSV queda no directorio do script e emítese unha advertencia.

Deste xeito mantéñense organizados por plataforma todos os resultados de execución para posterior análise ou comparativa.

Tamén pódese empregar o script `get_all_times.sh`, que compila todos os programas co script `compile_all.sh` e executa o script `tiempo_ejecucion.py` para todos os programas da arquitectura segundo os mesmos criterios que os scripts de execución.

Este script pode recibir tres flags:

- `-d|-directories` : fai que o script empregue unha lista personalizada de directorios que se ten que indicar a continuación do flag. Os directorios deberán estar separados por espazos e idealmente estar dentro do directorio **Programas**, pero acepta rutas relativas a este directorio.
- `-e|-extended` : fai que o script lle indique a `tiempo_ejecucion.py` que ten que empregar a lista estendida de valores de n .
- `-h|-help` : imprime a axuda do script.

B.4.2. Obtención dos resultados de execución

A obtención dos datos de execución realízase co programa `tiempo_ejecucion.py` que extrae os datos de execución dun programa dado e os almacena nun formato tratado, que varía segundo sexan matrices ou vectores.

Acepta varios resultados de execución por programa e detéctaos capturando a saída e buscando a liña “Resultado de ejecucion:” e recollendo o tempo que se imprime a continuación. A cantidade de tempos de execución detéctaos cunha execución inicial para un valor de n pequeno.

O script execútase deste xeito:


```
python extract_results.py -p <programa> -n <n_value> \
    [-s <seed>] [-q] [-m] [-o]
```

- p, -programa <ruta> Ruta ao programa cuxos resultados se queren extraer (obrigatorio).
- n, -n_value <int> Número de valores que xerou o programa; úsase para etiquetar e validar resultados (obrigatorio).
- s, -seed <int> Semente para a xeración de aleatorios. Valor por defecto: 1234.
- q, -use_qemu Se está presente, executa o programa emulando con `qemu-aarch64`.
- m, -matrix Activa modo matriz (interpreta *n* como dimensión de filas e columnas da matriz).
- o, -origin Indica que se extraen os datos antes da execución (extráese todo nunha mesa execución). Para atopar estes datos búscase a liña “Datos ejecucion:”.

Tras extraer os datos, o script xera e move os ficheiros CSV do seguinte xeito:

■ Xeración de CSVs:

- Datos “de orixe” (só se está activo -o e existen datos):
`source_<programa>_n<N>_seed<S>[_QEMU].csv`
- Resultados de execución (sempre que haxa datos):
`<programa>_n<N>_seed<S>[_QEMU].csv`
- `_QEMU` engádese se se usou emulación (-q).
- En modo matriz (-m) a cabeceira é `Ejecucion, Fila, Columna, Valor`; en array é `Ejecucion, Indice, Valor`.

■ Directorio destino:

```
/Metricas/Datos/Resultados-ejecucion/<vendor>/
```

- <vendor> obtense igual que para os tempos de execución.
- Ficheiros que comezan por `source_` van a
`../Resultados-ejecucion/<vendor>/Datos-previos-operaciones/`
- O resto queda en
`../Resultados-ejecucion/<vendor>/`

- Se a arquitectura non é xestionada, os CSV quedan no directorio do script e móstrase unha advertencia.

Tamén pódese empregar o script `get_all_results.sh`, que compila todos os programas co script `compile_all.sh` e executa o script `resultado_ejecucion.py` para todos os programas da arquitectura segundo os mesmos criterios que os scripts de execución.

Este script pode recibir tres flags:

- d|-directories : fai que o script empregue unha lista personalizada de directorios que se ten que indicar a continuación do flag. Os directorios deberán estar separados por espazos e idealmente estar dentro do directorio **Programas**, pero acepta rutas relativas a este directorio.
- q|-qemu : este flag só funciona en arquitectura x86. Fai que o script execute o `compile_all.sh` co flag `-f`. E tamén que, unha vez que remate a execución normal, volva executarse só para os programas con extensión `.out` e indicando a `resultado_ejecucion.py` que se executen empregando QEMU.
- h|-help : imprime a axuda do script.

B.4.3. Obtención das métricas

Para a obtención das métricas a partir dos datos obtidos anteriormente empréganse os seguintes programas do directorio **Metricas/Programas**:

speedup.py: Script que calcula o speedup entre dous ficheiros CSV de tempos base e mellorados.

- s/-source Ruta ao CSV con tempos orixinais.
- n/-new Ruta ao CSV con tempos mellorados.
- o/-output Nome base para o CSV de saída (sen extensión, por defecto `speedup.csv`).

Os ficheiros de entrada e saída deben seguir o formato de almacenamento de tempos de execución descrito anteriormente.

MAE.py: Script que calcula o Error Absoluto Medio (MAE) entre un CSV de referencia e varios CSV de comparación. Os argumentos son:

- true_file** Primeiro argumento posicional: ficheiro CSV de referencia.
- pred_files** Un ou varios argumentos posicionales seguintes: ficheiros CSV a comparar.
- t/-type Tipo de datos, obrigatorio: `vector` ou `matrix`.

-o/-output Nome base para o CSV de resultados (sen extensión). Se non se fornece, non se xera ficheiro de saída.

Os ficheiros de entrada e saída deben seguir o formato de almacenamento de datos de execución descrito anteriormente.

MSE.py: Script que calcula o Error Cuadrático Medio (MSE) entre un CSV de referencia e varios CSV de comparación. Emprega os mesmos argumentos e flags que **MAE.py** (**true_file**, **pred_files**, **-t/-type**, **-o/-output**). Os ficheiros de entrada e saída deben seguir o formato de almacenamento de datos de execución descrito anteriormente.

NRMSE.py: Script que calcula a Raíz do Erro Cadrático Medio Normalizada (NRMSE) entre un CSV de referencia e varios CSV de comparación. Emprega os mesmos argumentos e flags que **MAE.py** (**true_file**, **pred_files**, **-t/-type**, **-o/-output**). Os ficheiros de entrada e saída deben seguir o formato de almacenamento de datos de execución descrito anteriormente.

Pearson.py: Script que calcula o Coeficiente de Correlación de Pearson entre un CSV orixinal e un CSV procesado.

-s/-source Ruta ao CSV de referencia (FP32).

-p/-post-operations Ruta ao CSV procesado.

-t/-type Tipo de datos: **vector** ou **matrix**.

-o/-output Nome base para o CSV de saída (sen extensión).

Os ficheiros de entrada e saída deben seguir o formato de almacenamento de datos de execución descrito anteriormente.

Apéndice C

Licenza

MIT License

MIT License

Copyright (c) 2025 Eloi Barcón Piñeiro

Permission is hereby granted, free of charge, to any person
↪ obtaining a copy
of this software and associated documentation files (the
↪ "Software"), to deal
in the Software without restriction, including without limitation
↪ the rights
to use, copy, modify, merge, publish, distribute, sublicense,
↪ and/or sell
copies of the Software, and to permit persons to whom the Software
↪ is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be
↪ included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
↪ EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
↪ MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT
↪ SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR
↪ OTHER

LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE,
 ↪ ARISING FROM,
 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
 ↪ DEALINGS IN THE
 SOFTWARE.

<https://opensource.org/licenses/MIT>

LAPACK License

Copyright (c) 1992-2025 The University of Tennessee and The
 ↪ University

of Tennessee Research Foundation. All
 ↪ rights
 reserved.

Copyright (c) 2000-2025 The University of California Berkeley. All
 rights reserved.

Copyright (c) 2006-2025 The University of Colorado Denver. All
 ↪ rights

reserved.

\$COPYRIGHT\$

Additional copyrights may follow

\$HEADER\$

Redistribution and use in source and binary forms, with or without
 modification, are permitted provided that the following conditions

↪ are
 met:

- Redistributions of source code must retain the above copyright
 notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above
 ↪ copyright
 notice, this list of conditions and the following disclaimer
 ↪ listed
 in this license in the documentation and/or other materials
 provided with the distribution.

- Neither the name of the copyright holders nor the names of its contributors may be used to endorse or promote products derived
 ↪ from
 this software without specific prior written permission.

The copyright holders provide no reassurances that the source code provided does not infringe any patent, copyright, or any other intellectual property rights of third parties. The copyright
 ↪ holders
 disclaim any liability to any recipient for claims brought against recipient by any third party for infringement of that parties intellectual property rights.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
 ↪ CONTRIBUTORS
 "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS
 ↪ FOR
 A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE
 ↪ COPYRIGHT
 OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT,
 ↪ INCIDENTAL,
 SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF
 ↪ USE,
 DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON
 ↪ ANY
 THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR
 ↪ TORT
 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE
 ↪ USE
 OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH
 ↪ DAMAGE.

<https://github.com/Reference-LAPACK/lapack/blob/master/LICENSE>

Bibliografía

- [1] Institute of Electrical e Electronics Engineers. *IEEE Standard for Floating-Point Arithmetic*. Standard IEEE Std 754-2008. Revision of IEEE Std 754-1985. New York, NY, USA: Institute of Electrical e Electronics Engineers, 29 de ago. de 2008. DOI: 10.1109/IEEESTD.2008.4651431. URL: <http://standards.ieee.org/findstds/standard/754-2008.html>.
- [2] Wikipedia contributors. *Bfloat16 Floating-Point Format*. Sen data. URL: https://en.wikipedia.org/wiki/Bfloat16_floating-point_format (accedido en 11/06/2025).
- [3] X. Zhang et al. “A Study of BFLOAT16 for Deep Learning Training”. En: *arXiv preprint arXiv:1905.12322* (2019). URL: <https://arxiv.org/pdf/1905.12322.pdf> (accedido en 23/06/2025).
- [4] Wikipedia contributors. *AArch64 – ARMv8.2-A*. Sen data. URL: <https://es.wikipedia.org/wiki/AArch64#ARMv8.2-A> (accedido en 05/06/2025).
- [5] Universidade de Santiago de Compostela (USC) e Escola Técnica Superior de Enxeñaría (ETSE). *Regulamento do Traballo Fin de Grao: Grao en Enxeñaría Informática*. Normativa oficial. Documento aprobado polo Consello de Goberno da USC na sesión ordinaria do día 30 de xaneiro de 2025 e pola Xunta de Escola da ETSE o 29 de novembro de 2024. CSV: E49D-E52E-404E-9F39. Xan. de 2025. URL: <https://sede.usc.es/csv/E49D-E52E-404E-9F39> (accedido en 06/06/2025).
- [6] Wikipedia contributors. *F16C*. Sen data. URL: <https://en.wikipedia.org/wiki/F16C> (accedido en 05/06/2025).
- [7] Daniel Towner et al. *Intel® AVX-512 - FP16 Instruction Set for Intel® Xeon® Processor-Based Products*. Intel Corporation, mar. de 2024. URL: <https://builders.intel.com/docs/networkbuilders/intel-avx-512-fp16-instruction-set-for-intel-xeon-processor-based-products-technology-guide-1710148129.pdf> (accedido en 06/06/2025).
- [8] Cellular Stockpile. *AMD Takes a Leap Towards AI in Their CPUs: Zen 5 and Zen 6 to Support New AVX512 Instructions like Intel*. Tech Article. 9 de xan. de 2024. URL: <https://cellularstockpile.com/amd-takes->

- a-leap-towards-ai-in-their-cpus-zen-5-and-zen-6-to-support-new-avx512-instructions-like-intel/ (accedido en 07/06/2025).
- [9] Protector Indefinido. *AMD da un paso más hacia la IA en sus CPU: Zen 5 y Zen 6 soportarán nuevas instrucciones AVX512 como Intel*. Tech Article. 8 de xan. de 2024. URL: <https://elchapuzasinformatico.com/2024/01/amd-cpu-zen-5-avx512-vp2intersect-zen-6-avx512-fp16/> (accedido en 07/06/2025).
 - [10] Wikipedia contributors. *AVX-512: CPUs with AVX-512*. Sen data. URL: https://en.wikipedia.org/wiki/AVX-512#CPUs_with_AVX-512 (accedido en 08/06/2025).
 - [11] Xaver Amberger (skullbringer). *Intel Deactivated AVX-512 on Alder Lake but Fully Questionable Interpretation of Efficiency – News, Editorial*. 31 de dec. de 2021. URL: <https://www.igorslab.de/en/intel-deactivated-avx-512-on-alder-lake-but-fully-questionable-interpretation-of-efficiency-news-editorial/> (accedido en 08/06/2025).
 - [12] Aaron Klotz. *How to Tell Which Alder Lake CPUs Have AVX-512*. 25 de abr. de 2022. URL: <https://www.tomshardware.com/news/how-to-tell-which-alder-lake-cpus-have-avx-512> (accedido en 08/06/2025).
 - [13] Alberto Villarreal Cueva. *Code Sample: Intel® Deep Learning Boost New Deep Learning Instruction bfloat16 - Intrinsic Functions*. 18 de xuñ. de 2020. URL: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-deep-learning-boost-new-instruction-bfloat16.html> (accedido en 08/06/2025).
 - [14] Intel Corporation. *Intel® Architecture Instruction Set Extensions and Future Features Programming Reference*. Programming Reference Document 319433-038. Santa Clara, CA, USA: Intel Corporation, mar. de 2020. URL: <https://www.intel.com/content/dam/develop/external/us/en/documents/architecture-instruction-set-extensions-programming-reference-737410.pdf>.
 - [15] Wikipedia contributors. *AArch64*. Sen data. URL: <https://es.wikipedia.org/wiki/AArch64> (accedido en 08/06/2025).
 - [16] Arm Developer. *BFloat16 Floating-Point Number Format*. Sen data. URL: <https://developer.arm.com/documentation/101754/0624/armclang-Reference/Other-Compiler-specific-Features/BFloat16-floating-point-number-format> (accedido en 11/06/2025).
 - [17] RISC-V Foundation. *RISC-V Instruction Set Manual, Volume I: RISC-V User-Level ISA*. Release: riscv-isa-release-1239329-2023-05-23-96-g1ee25e1. 27 de set. de 2023. URL: <https://www.five-embeddev.com/riscv-user-isa-manual/latest-adoc/zfh.html> (accedido en 08/06/2025).

- [18] Kenneth Dockser (kdockser). *Extensions*. Fichero no repositorio de GitHub riscv-bfloat16. 16 de dec. de 2023. URL: <https://github.com/riscv/riscv-bfloat16/blob/main/doc/riscv-bfloat16-extensions.adoc> (accedido en 08/06/2025).
- [19] RISC-V Community. *RISC-V BF16 Extensions*. Colaboradores a esta versión da especificación: GouYue, Ken Dockser (Editor), Kenneth Rovers, Nick Knight e Nicolas Brunie. 5 de xul. de 2024. URL: <https://mirror.iscas.ac.cn/riscv-toolchains/release/riscv/riscv-bfloat16/LatestRelease/riscv-bfloat16.pdf> (accedido en 08/06/2025).
- [20] Michael Larabel. *RISC-V With Linux 6.15 Adds Support For BFloat16 "BF16" Instructions*. Artículo de Phoronix sobre el soporte para BF16 en RISC-V en el kernel Linux 6.15. 5 de abr. de 2025. URL: <https://www.phoronix.com/news/Linux-6.15-RISC-V> (accedido en 10/06/2025).
- [21] Wikipedia contributors. *Power ISA v.3.1*. Sen data. URL: https://en.wikipedia.org/wiki/Power_ISA#Power_ISA_v.3.1 (accedido en 09/06/2025).
- [22] Wikipedia contributors. *Power10*. Sen data. URL: <https://en.wikipedia.org/wiki/Power10> (accedido en 09/06/2025).
- [23] Loongson Technology Corporation Limited. *LoongArch Reference Manual - Volume 1: Basic Architecture*. 11 de abr. de 2024. URL: <https://github.com/loongson/LoongArch-Documentation/releases/latest/download/LoongArch-Vol1-v1.10-EN.pdf> (accedido en 09/06/2025).
- [24] Paul Jansen. *TIOBE Index*. Xuñ. de 2025. URL: <https://www.tiobe.com/tiobe-index/> (accedido en 10/06/2025).
- [25] Wikipedia contributors. *GNU Compiler Collection*. Sen data. URL: https://en.wikipedia.org/wiki/GNU_Compiler_Collection (accedido en 10/06/2025).
- [26] Anton Shilov. *GCC Gains Loongson's LoongArch Architecture Support*. 18 de maio de 2022. URL: <https://www.tomshardware.com/news/gcc-now-supports-loongson-loongarch> (accedido en 10/06/2025).
- [27] GNU Project. *Floating Types*. GNU Compiler Collection Documentation. URL: <https://gcc.gnu.org/onlinedocs/gcc/Floating-Types.html> (accedido en 10/06/2025).
- [28] *ISO/IEC TS 18661-3:2015 — Floating-point extensions for C — Part 3: Interchange and extended types*. ISO/IEC. Out. de 2015. URL: <https://www.iso.org/standard/65615.html> (accedido en 10/06/2025).
- [29] Michael Larabel. *LLVM Clang 14 Begins Landing Intel AVX-512 FP16 Support*. 16 de ago. de 2021. URL: <https://www.phoronix.com/news/LLVM-Clang-AVX512FP16> (accedido en 10/06/2025).

- [30] GNU Project. *Half-Precision Floating Point*. Sen data. URL: <https://gcc.gnu.org/onlinedocs/gcc/Half-Precision.html> (accedido en 10/06/2025).
- [31] The GCC Documentation Team. *-fexcess-precision (Optimize Options)* — Using the GNU Compiler Collection (GCC). Sen data. URL: <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html#index-fexcess-precision> (accedido en 13/06/2025).
- [32] The GCC Documentation Team. *x86 Options (Using the GNU Compiler Collection (GCC))*. Sen data. URL: <https://gcc.gnu.org/onlinedocs/gcc-14.2.0/gcc/x86-Options.html> (accedido en 11/06/2025).
- [33] System V ABI Committee e hongtao. *x86-64 ABI: Low-Level System Characteristics (low-level-sys-info.tex)*. Commit 81957300 que engade soporte para `__bf16`. 21 de xuñ. de 2022. URL: <https://gitlab.com/x86-psABIs/x86-64-ABI/-/blob/81957300/x86-64-ABI/low-level-sys-info.tex> (accedido en 10/06/2025).
- [34] Michael Larabel. *GCC & LLVM Ready With x86 `__bf16` Type Support*. 18 de ago. de 2022. URL: <https://www.phoronix.com/news/GCC-LLVM-bf16-BFloat16-Type> (accedido en 10/06/2025).
- [35] GNU Project. *AArch64 Options: march and -mcpu Feature Modifiers*. Sen data. GNU Compiler Collection Documentation. URL: https://gcc.gnu.org/onlinedocs/gcc/AArch64-Options.html#g_t-march-and--mcpu-Feature-Modifiers (accedido en 10/06/2025).
- [36] The GCC Documentation Team. *GCC 9 Release Series — Changes, New Features, and Fixes* — Using the GNU Compiler Collection (GCC). 27 de maio de 2022. URL: <https://gcc.gnu.org/gcc-9/changes.html> (accedido en 23/06/2025).
- [37] Arm Limited. *Advanced SIMD (Neon) intrinsics*. Arm Developer. Xul.-set. de 2020. URL: <https://developer.arm.com/documentation/101028/0012/13--Advanced-SIMD--Neon--intrinsics> (accedido en 10/06/2025).
- [38] The GCC Documentation Team. *GCC 14 Changes* — Using the GNU Compiler Collection (GCC). 23 de maio de 2025. URL: <https://gcc.gnu.org/gcc-14/changes.html> (accedido en 13/06/2025).
- [39] The GCC Documentation Team. *GCC 10 Release Series — Changes, New Features, and Fixes* — Using the GNU Compiler Collection (GCC). 7 de xul. de 2023. URL: <https://gcc.gnu.org/gcc-10/changes.html> (accedido en 23/06/2025).
- [40] Wikipedia contributors. *Clang*. Sen data. URL: <https://en.wikipedia.org/wiki/Clang> (accedido en 11/06/2025).

- [41] LLVM Project. *Language Extensions: Half-Precision Floating Point*. Sen data. URL: <https://clang.llvm.org/docs/LanguageExtensions.html#half-precision-floating-point> (accedido en 11/06/2025).
- [42] Michael Larabel. *LLVM 15.0 Released With Security Features, Experimental DirectX Back-End, New AMD GPUs*. 6 de set. de 2022. URL: <https://www.phoronix.com/news/LLVM-15.0-Released> (accedido en 11/06/2025).
- [43] LLVM Project. *Clang Command Line Reference: -mavx512fp16*. Sen data. URL: <https://clang.llvm.org/docs/ClangCommandLineReference.html#cmdoption-clang-mavx512fp16> (accedido en 11/06/2025).
- [44] LLVM Project. *Saída do comando clang -cc1 -help e extracción da opción fnative-half-arguments-and-returns*. Obtida executando `clang -cc1 -help | grep -i -A 1 "fnative-half-arguments-and-returns"` na versión 18.1.3 para Ubuntu de Clang. LLVM Project. URL: <https://llvm.org/> (accedido en 13/06/2025).
- [45] LLVM Project. *Clang Command Line Reference: -mavx512bf16*. Sen data. URL: <https://clang.llvm.org/docs/ClangCommandLineReference.html#cmdoption-clang-mavx512bf16> (accedido en 11/06/2025).
- [46] Arm Developer. *armclang Command Line Options: -march*. Sen data. URL: <https://developer.arm.com/documentation/101754/0624/armclang-Reference/armclang-Command-line-Options/-march?lang=en> (accedido en 11/06/2025).
- [47] Arm Developer. *armclang Reference: Other Compiler-specific Features – Supported Architecture Features: Floating-point Extensions*. Sen data. URL: <https://developer.arm.com/documentation/101754/0624/armclang-Reference/Other-Compiler-specific-Features/Supported-architecture-features/Floating-point-extensions?lang=en> (accedido en 11/06/2025).
- [48] Arm Developer. *Half-precision floating-point data types*. Sen data. URL: <https://developer.arm.com/documentation/101754/0624/armclang-Reference/Other-Compiler-specific-Features/Half-precision-floating-point-data-types?lang=en> (accedido en 11/06/2025).
- [49] Matthias Kretz e ISO C++ Foundation. *Floating-Point Excess Precision*. Paper number P3488R0; Target C++ 26. 14 de nov. de 2024. URL: <https://isocpp.org/files/papers/P3488R0.pdf> (accedido en 11/06/2025).
- [50] Wikipedia contributors. *Half-precision floating-point format — Wikipedia*. Sen data. URL: https://en.wikipedia.org/wiki/Half-precision_floating-point_format (accedido en 11/06/2025).
- [51] Jason Andrews e Arm Inc. *Cross-Compilation with GCC: Install Guides — Arm Learning*. 5 de feb. de 2025. URL: <https://learn.arm.com/install-guides/gcc/cross/> (accedido en 15/06/2025).